HookSuite

Herramienta de auditoría de seguridad web con Inteligencia Artificial

Módulo · Máster en Ciberseguridad & IA | Curso 2026

Práctica · Práctica 3 — Del prototipo al producto

Grupo · Feet Lovers — sin número asignado

EQUIPO DE DESARROLLO

P1 **Ivan Medina**

P2 **Macarena Rogerio**

P3 **Nacho García**

P4 **Carlos Bañuelos**

P5 **José María López**

ÁREAS DE TRABAJO

Asumidas en común y sin reparto por persona (apartado 10).

Área	Qué se hizo
Frontend	Pulido del Spider y del Repeater, revisión de pantallas, nueva pestaña Manual y retirada de Red
Backend	Alta y acceso por usuario, aislamiento de datos y bus interno de mensajes
IA	Integración del análisis de hallazgos con el panel y control del gasto
Seguridad	Cierre de la API, orígenes permitidos, canal autenticado, gestión de secretos y limpieza del historial
Infraestructura y despliegue	Contenedores, servidor público y despliegue reproducible
Pruebas y evidencias	Plan de pruebas, ejecución y capturas
Documentación	Memoria técnica, README y vídeo

---

github.com/Feet-Lovers/Proyecto-Evolve

Entrega · 16 de octubre de 2026

Vídeo de demostración · PENDIENTE DE ENLACE

Índice

1. Resumen ejecutivo 4
2. Punto de partida 5
   2.1. Qué nos encontramos 5
   2.2. La lección más cara de la Práctica 1 5
   2.3. Numeración de requisitos 5
   2.4. Clasificación inicial de requisitos 6
   2.5. Sobre el feedback de la Práctica 1 6
3. Requisitos 7
   3.1. Requisitos funcionales (RF) 7
   3.2. Requisitos no funcionales (RNF) 8
   3.3. Requisitos modificados o descartados (con justificación) 8
4. Arquitectura y decisiones técnicas 11
   4.1. Componentes 11
   4.2. Enrutado 12
   4.3. Decisiones técnicas 12
5. Funcionalidades implementadas 14
   5.1. Las funcionalidades, una a una: qué hace, cómo se usa y qué requisito cubre . 14
   5.2. Nada de lo que se entrega funciona con datos simulados 16
6. Seguridad del producto 18
   6.1. Secretos: el incidente de las claves 18
   6.2. Exposición de la caja: cómo estaba 18
   6.3. Endurecimiento aplicado al inicio de la práctica (5-oct) 18
   6.4. Autenticación y aislamiento: de cuatro fugas a cero (8-oct) 19
7. Pruebas y evidencias 26
   7.1. El plan de pruebas: los dieciocho casos que cita la matriz 26
   7.2. Evidencia de exposición (RNF-07 / PR-16), capturada el 4-oct 27
   7.3. Los doce defectos del prototipo, corregidos y verificados 28
   7.4. Defectos corregidos: antes y después (RF-03) 28
   7.5. El pulido del Spider (11-oct): quince defectos, y catorce los encontró el uso 29
   7.6. El pulido del Repeater (12-oct): dieciséis defectos, y una función retirada el mismo día 30
   7.7. Condición de carrera detectada antes de reiniciar el servidor 32
   7.8. Otras evidencias ya en mano 33
   7.9. El aislamiento (RF-12): las fugas capturadas antes, y la verificación después . . 33
   7.10. El despliegue habría terminado «en verde» sin aplicar la configuración del proxy 34
   7.11. La instalación desde cero, ejecutada: dos defectos que solo aparecen al seguir las instrucciones 35
   7.12. Dos defectos de montaje que impedían operar 35
   7.13. Un módulo que parecía sano y no podía hablar con nadie (RF-08) 35
   7.14. Las dos contrapartidas del bus, probadas antes de confiar en él 36
   7.15. El recorrido completo del disparador, verificado de punta a punta (RF-08) 36
   7.16. El techo de gasto, ejecutado: una auditoría entera por 0 € (RF-08) 36

7.17. Operación degradada (RNF-06), probada sin gastar saldo 37
7.18. RNF-06 visto en el panel, que es donde tenía que verse 37
7.19. Lo que el modelo puede devolver, y lo que el código daba por hecho (RF-08) . . 39
7.20. Tres rechazos seguidos de la API: validar contra tu propia prueba no es validar (RF-08) 39
7.21. Un fallo silencioso no es un fallo menor: la escala de la confianza (RF-08 / RNF-06) 40
7.22. La interfaz antes de descartar RF-09, capturada antes de retirarla 40
8. Matriz de trazabilidad 41
9. Limitaciones y trabajo futuro 46
9.1. El estado de sesión es volátil, por diseño 46
9.2. El techo de gasto de la IA: aplicado, y lo que sigue sin estar acotado 46
9.3. Lo que queda de la confusión de escala, una vez cerrada 47
9.4. El manual de uso va por dos pestañas escritas y tres por escribir 47
9.5. Dos cosas del Spider que se dejan dichas, no hechas 48
9.6. El Repeater trabaja con una petición cada vez, y es una decisión 48
9.7. El borrador del Repeater guarda las credenciales del objetivo en el navegador . 49
9.8. Lo que falta para igualar al Repeater de referencia 50
9.9. El frontend no pasa su propio analizador estático 50
9.10. Restos del laboratorio retirado 50
9.11. La herramienta que genera los esquemas no está versionada 50
10. Reparto del trabajo 52
11. Uso de herramientas de IA 55
11.1. Los límites de la herramienta, observados en primera persona 55
11.2. Los defectos que introdujo el asistente al arreglar otros 55
11.3. Leer sin leer: delegar la lectura de lo que no debe entrar en la conversación . . . 56
11.4. Qué aportó el asistente, fase por fase 57
11.5. Generado frente a verificado: la regla, y lo que cuesta sostenerla 58
11.6. Lo que el registro de horas no puede decir 58
12. Anexos 59
12.1. Manual de despliegue 59
12.2. Usuarios de prueba y acceso para la corrección 60
12.3. Glosario 60
12.4. Referencias 61

SECCIÓN 1

Resumen ejecutivo

HookSuite es una herramienta de auditoría de seguridad web que se usa desde el navegador, sin instalar nada en el equipo del auditor. Reúne en un solo panel el rastreo de un sitio, el reenvío manual de peticiones, la prueba masiva de valores sobre un parámetro, un juego de utilidades de codificación y un módulo de inteligencia artificial que clasifica los hallazgos.

Para quién. Para quien audita aplicaciones web y no quiere depender de una herramienta instalada en su equipo. Se abre una dirección, se entra con un usuario propio y se trabaja con datos aislados de los del resto de cuentas.

De dónde viene. La Práctica 1 dejó un prototipo que funcionaba, pero que no era defendible. La verificación en vivo del 29 de septiembre lo dejó claro: la API respondía desde internet sin pedir credenciales, la credencial documentada no abría el panel y cuatro claves de la API de Anthropic estaban dentro del historial de git. Esta práctica parte de ahí y lleva el prototipo a un estado instalable, desplegado y verificable. El recorrido completo está en el apartado 2.

En qué estado se entrega. De los veintiún requisitos numerados, trece quedan cumplidos, cuatro parciales, dos modificados, uno descartado y uno pendiente.

Los cuatro parciales conviene decirlos uno a uno. El módulo de navegador está construido y conectado a la red interna, pero su prueba no se ha llegado a ejecutar. El historial del repositorio no refleja un reparto de tareas, sencillamente porque no lo hubo: la autoría de cada confirmación se sortea, y el apartado 10 lo explica. El análisis con inteligencia artificial tiene el recorrido verificado de punta a punta y el techo de gasto ejecutado, pero todavía no ha llegado a mostrar un hallazgo analizado, porque todas las auditorías que han terminado el recorrido han salido degradadas. Y la seguridad del propio producto queda parcial por las dependencias: acumulan avisos conocidos y en esta práctica no se actualizó ninguna, el apartado 6 da los números.

El requisito pendiente es el cifrado del tráfico, así que el producto se sirve hoy por HTTP. Lo demás funciona de principio a fin: alta y acceso por usuario con datos aislados entre cuentas, rastreo del objetivo, reenvío manual de peticiones, prueba masiva de valores sobre un parámetro y utilidades de codificación.

SECCIÓN 2

Punto de partida

HookSuite nació en la Práctica 1 como prototipo de una suite de auditoría web al estilo de Burp Suite, operada desde el navegador y asistida por inteligencia artificial. Quedó desplegado en un servidor de Hetzner con Docker Compose y, desde la entrega de mayo de 2026, nadie volvió a tocarlo. La Práctica 3 lo retoma tal como quedó.

Qué nos encontramos

La verificación en vivo del 29 de septiembre y del 4 de octubre dio este cuadro.

- La herramienta seguía en pie desde mayo y sin cambios. El dominio resolvía al servidor de Hetzner, y respondían tanto el servidor web como la API, con sus treinta y seis rutas.
- El núcleo funcionaba: rastreo del objetivo, reenvío manual de peticiones, prueba masiva de valores sobre un parámetro, utilidades de codificación, sesiones por token y canal de actualización en vivo.
- La API respondía en el puerto 8000 abierta a internet y sin ninguna autenticación, incluidas las rutas que lanzan tráfico contra terceros.
- En el historial de git había claves de API reales.
- El módulo de inteligencia artificial, que es lo que da sentido al producto, nunca llegó a operar más allá de unas pruebas iniciales. El panel estaba construido, pero inactivo.

Los dos hallazgos de seguridad reorientaron el trabajo de esta práctica y se tratan en el apartado 6.

La lección más cara de la Práctica 1

En las primeras pruebas del módulo de inteligencia artificial se cargaron cinco euros de saldo en la API y se agotaron en muy poco tiempo. La causa, reconstruida del historial de git, fue que el módulo ejecutaba una auditoría completa al arrancar y el servicio estaba declarado para reiniciarse solo, de modo que cada vez que el proceso terminaba volvía a empezar otra auditoría. Se corrigió en mayo de 2026 haciendo que el módulo espere instrucciones en lugar de auditar al arrancar.

Figura en el punto de partida porque esta práctica hereda el problema a medias. El disparo automático quedó curado, pero el tamaño de una auditoría siguió sin acotarse, y esa es una de las razones por las que el módulo llegó hasta aquí sin haber operado. El techo de gasto que lo resuelve se explica en los apartados 7 y 9.

Numeración de requisitos

La Práctica 1 no numeraba los requisitos. El enunciado de la Práctica 3 obliga a numerarlos como RF-xx y RNF-xx y a usar esa numeración en toda la práctica. La lista completa está en el apartado 3 y está derivada del informe de la Práctica 1, de lo que el producto prometía y de lo que realmente hacía.

Clasificación inicial de requisitos

Con las cinco categorías que define el enunciado. Esta tabla es la foto del punto de partida y no se ha retocado después.

Estado	Qué significa	Requisitos
Cumplido	Implementado en la P1 y funciona. En la P3 se verifica con prueba y evidencia	RF-01, RF-03, RF-04, RF-05, RF-06, RF-07, RF-11 · RNF-01, RNF-02, RNF-03, RNF-05, RNF-08 **(12)**
Parcial	Existe algo, pero incompleto o con fallos. Hay que terminarlo y probarlo	RF-08 · RNF-04, RNF-07 **(3)**
Pendiente	Definido, pero sin implementar. Se hace desde cero	RF-09, RF-10, RF-12 · RNF-09 **(4)**
A revisar	El feedback o la experiencia indican que estaba mal planteado. Se reformula con justificación	RF-02 · RNF-06 **(2)**
Descartado	Se decide no implementarlo. Debe ser la excepción y justificarse	ninguno

Dos requisitos cambiaron de estado más adelante, RF-04 y RF-09, y ese cambio se justifica en el apartado 3.

Sobre el feedback de la Práctica 1

El enunciado pide reunir la calificación y los comentarios del equipo docente y construir esta práctica sobre ellos. El grupo no recibió ni calificación ni comentarios, así que ese material no existe y no se puede citar. Se dice aquí en lugar de dejar el apartado a medias.

Las mejoras salen entonces de dos fuentes propias. La primera es el plan de mejoras que la propia Práctica 1 dejó escrito, del que se abordaron los cinco primeros puntos. La segunda es la revisión crítica del producto hecha ejecutándolo, que encontró los defectos que esta memoria documenta y corrige uno a uno. Los dos más graves ya están dichos: la API respondía sin credenciales y la credencial documentada no abría el panel.

Una precisión sobre la tabla. La categoría A revisar se define como aquella en la que el feedback o la experiencia indican que el requisito estaba mal planteado. Sin feedback, en esta práctica fue siempre la experiencia, y por eso RF-02 y RNF-06 se reformularon por lo que se vio al usar el producto.

SECCIÓN 3

Requisitos

Lista completa y numerada. La numeración se fijó en esta práctica, porque la Práctica 1 no la tenía, y se derivó de su informe. La columna de estado parte de la clasificación inicial del apartado 2 y recoge los cambios que esta práctica fue introduciendo.

Requisitos funcionales (RF)

ID	Requisito	Estado (de partida, puesto al día durante la P3)
RF-01	Dashboard web accesible desde cualquier navegador, sin instalar nada en el cliente	Cumplido
RF-02	El servidor ejecuta las peticiones HTTP en nombre del auditor (proxy del lado servidor)	**Modificado**, justificado abajo
RF-03	Spider: rastreo automático del objetivo (BFS, scope al dominio, extrae formularios), velocidad configurable	Cumplido
RF-04	Auditoría autenticada: sesión httpx persistente compartida por Spider/Repeater/Intruder que hereda las cookies del login	**Modificado**, justificado abajo
RF-05	Repeater: modificar y reenviar cualquier petición, con respuesta en 4 vistas (Raw/Pretty/Preview/Headers)	Cumplido
RF-06	Intruder: fuzzing de parámetros marcados con un asterisco, 4 tipos (SQLi, Blind SQLi, XSS, genérico), con resultados en tiempo real	Cumplido
RF-07	Utilidades: Encoder/Decoder (Base64/URL/HTML/JWT), Hash (MD5/SHA1/SHA256/SHA512), Regex Tester, Payload Generator	Cumplido
RF-08	**Análisis y clasificación de vulnerabilidades con IA (Claude) según OWASP** → panel Vulnerabilidades	Parcial
RF-09	Captura de tráfico HTTP en tiempo real (DevTools / CDP) → panel Red con código de colores	**Descartado**, justificado abajo
RF-10	Ejecución de ataques con navegador real (Playwright)	**Parcial**, construido y en la red interna, sin prueba ejecutada (apartados 6 y 12)
RF-11	Soporte multi-usuario simultáneo mediante tokens de sesión UUID	Cumplido
RF-12	Registro y login individual con JWT + aislamiento de historial/auditorías por usuario	Cumplido (mejora, no venía de la P1)

> **Matiz sobre el enunciado de RF-03.** El requisito habla de «velocidad configurable». El control que existe no regula velocidad: fija un tope de páginas a visitar. Un valor menor no rastrea más rápido, llega menos lejos. El enunciado se conserva tal como se escribió en la P1,

porque no se reescriben requisitos a posteriori, pero la interfaz sí se corrigió para decir lo que de verdad hace, y el control pasó a llamarse «hasta N páginas».

Requisitos no funcionales (RNF)

ID	Requisito	Estado (de partida, puesto al día durante la P3)
RNF-01	Accesible desde navegador sin instalar software en el cliente	Cumplido
RNF-02	Actualización en tiempo real vía WebSocket (pings cada 30 s, eventos {type,payload})	Cumplido
RNF-03	Desplegado en servidor Hetzner con contenedores Docker	Cumplido
RNF-04	Código en GitHub con historial de commits que refleje el trabajo del grupo	Parcial
RNF-05	Estabilidad ante carga: límite de concurrencia para no saturar producción	Cumplido
RNF-06	Operación degradada cuando la IA no responde o no supera el umbral de confianza	**Cumplido**, justificado abajo
RNF-07	**Seguridad del propio producto**: secretos fuera del repo, control de acceso, validación de entradas, dependencias sin CVEs	Parcial
RNF-08	Firewall dinámico por sesión (abre un puerto por sesión, lo cierra al logout o tras 4 h)	Cumplido en la P1 y endurecido después, pero **no reejercitado en esta práctica**
RNF-09	TLS/HTTPS en los endpoints públicos (Let's Encrypt)	Pendiente

Requisitos modificados o descartados (con justificación)

El enunciado pide que todo requisito modificado o descartado lleve su versión original, la nueva, el motivo y las consecuencias.

RF-02 — proxy del lado servidor (modificado)

- **Versión original:** interceptación del tráfico del navegador mediante archivo de autoconfiguración de proxy (PAC) y WebSockets.
- **Versión nueva:** el servidor ejecuta las peticiones directamente con httpx, sin interponerse en el navegador del auditor.
- **Motivo.** En la P1 el modelo PAC quedó expuesto y se saturó con tráfico de bots. El pivote a httpx directo elimina esa superficie y simplifica el flujo.
- **Consecuencias.** El fichero de autoconfiguración se retiró del código, junto con su ruta en el backend y sus dos bloques en Nginx, después de comprobar que ya no lo usaba nadie. Con él desaparece la única ruta que tenía que quedar sin autenticar, porque un navegador no envía credenciales al pedirla, de modo que ahora toda la API exige token sin excepciones.

RF-04 — auditoría autenticada con sesión compartida (modificado)

- **Versión original:** tal como está escrito el requisito, una sesión httpx persistente compartida por Spider, Repeater e Intruder, que hereda las cookies del login del objetivo.

- **Versión nueva:** el Repeater deja de usar esa sesión compartida y envía una petición independiente, con exactamente las cabeceras que se ven en pantalla. Spider, Proxy e Intruder siguen compartiéndola.
- **Motivo.** El requisito describe un mecanismo, pero lo que persigue es poder auditar autenticado, y ese mecanismo rompía en el Repeater algo que vale más, la reproducibilidad. El cliente compartido enviaba cookies que no estaban en la petición y se quedaba con las que devolvía el servidor, así que dos envíos idénticos daban resultados distintos y lo que se veía en pantalla no era lo que salía por el cable.
- **Consecuencias.** El requisito se sigue cumpliendo, porque la sesión se hereda igual pero por un canal explícito: las cabeceras que el producto muestra son las realmente enviadas, de modo que la cookie del objetivo viaja en la petición, visible y editable. El cambio destapó además un defecto anterior. Al quitar el cliente compartido, todos los envíos por POST empezaron a devolver 419, porque los formularios que el Spider descubre viajaban con las cabeceras incompletas y el cliente compartido lo estaba tapando. Se corrigió el origen y el fallo desapareció.

RNF-06 — operación degradada (a revisar en la P1, cumplido y demostrado en la P3)

- **Versión original:** enunciado como principio en la P1, sin demostrar.
- **Versión nueva:** la operación degradada se demuestra haciendo que un fallo de la IA sea distinguible de un análisis correcto. El clasificador devuelve tres estados donde antes había dos: vulnerable con confianza suficiente, analizado y limpio, y no analizado acompañado del motivo.
- **Motivo.** Cuando la IA no podía analizar, el clasificador devolvía lo mismo que cuando había analizado y no encontraba nada, de modo que el panel decía «sin vulnerabilidades» sin distinguir un resultado de una ignorancia. Y si la llamada a la API fallaba, nadie capturaba la excepción y se interrumpía la fase de análisis entera. Lo uno no es operación degradada sino silencio, y lo otro no es degradación sino caída.
- **Consecuencias.** Los tres estados existen en el código y una prueba automática los recorre en las cuatro vías de análisis, sin clave de API ni red y con coste cero. El estado degradado se ve en el panel, separado de los hallazgos y con su motivo, como muestra la figura del apartado 7.

RF-12 — registro y login individual con JWT (mejora, no requisito P1)

- **Versión original:** no existía. No era requisito de la Práctica 1, venía del plan de mejoras que esa misma práctica dejó escrito.
- **Versión nueva:** registro y acceso individual con JWT, con los datos de cada cuenta aislados del resto.
- **Motivo.** Se aborda por su valor de seguridad, porque es lo que cierra la fuga de datos entre usuarios documentada en los apartados 6 y 7. Cuenta como mejora, que suma pero no compensa un requisito original sin cumplir.
- **Consecuencias.** La persona se da de alta ella misma y elige su propia contraseña. Nadie más la conoce, ni siquiera quien administra, y ese es justo el punto del requisito. Lo que hace falta además es un código de invitación que reparte el grupo. La razón es el propio producto: HookSuite lanza tráfico contra terceros, así que con altas anónimas cualquiera podría usar el Spider y el Intruder contra quien quisiera desde nuestra infraestructura. El código es obligatorio por configuración: si faltara, el servicio no arranca, en lugar de arrancar con el registro abierto sin que nadie lo note.

RF-09 — captura de tráfico en tiempo real con DevTools/CDP (descartado)

- **Versión original:** captura del tráfico HTTP en tiempo real mediante el protocolo de depuración de Chrome (CDP), volcada a un panel «Red» con código de colores por método y estado.
- **Versión nueva:** se descarta. El 11 de octubre se retiró del producto la pestaña Red, con su entrada de navegación, su ruta y su página, y la barra pasó de seis pestañas a cinco. Después se añadió la pestaña Manual, así que el producto que se entrega vuelve a tener seis: Spider, Repeater, Intruder, Utilidades, Vulnerabilidades y Manual.
- **Motivo.** Verificado contra el código antes de decidir: RF-09 no era una pantalla a medio pulir sino un vertical entero sin cerrar, con cuatro cosas rotas a la vez. El capturador se lanza a mano y busca el navegador en rutas de Windows que no existen en los contenedores, que son Linux. Sus hallazgos de consola se envían a un extremo que no existe en el backend. Su lanzador apunta a una dirección y un puerto que ya no están, y a la ruta del fichero PAC que se jubiló al reformular RF-02. Y la pestaña del panel era pasiva, no pedía nada al servidor, de modo que el usuario no podía distinguir «no hay tráfico» de «el capturador no está corriendo». Cerrarlo exigía arreglar las cuatro cosas y, sobre todo, probarlo, con el plazo de entrega encima.
- **Consecuencias.** El capturador permanece en el repositorio como punto de partida para más adelante. Los extremos de red del backend también se conservan, y no por inercia: los consume el módulo de IA y el de Playwright, así que retirarlos habría roto RF-08. RF-01, el acceso desde el navegador sin instalar nada, se mantiene cumplido, y la figura del apartado 7 documenta la pestaña Red tal como estaba antes de retirarla, capturada a propósito antes del arreglo porque el propio arreglo destruía la evidencia.

SECCIÓN 4

Arquitectura y decisiones técnicas

HookSuite se despliega como un conjunto de contenedores Docker coordinados por docker--compose, todos en una red interna hooksuite-net, con Nginx como única puerta de entrada. La arquitectura se verificó fichero a fichero sobre docker-compose.yml y infra/nginx.conf. El producto vive en dos instalaciones iguales, una de desarrollo y otra pública, y la pública solo recibe despliegues: nada se prueba sobre la instalación que usan las personas, porque esta herramienta lanza tráfico contra sistemas de terceros.

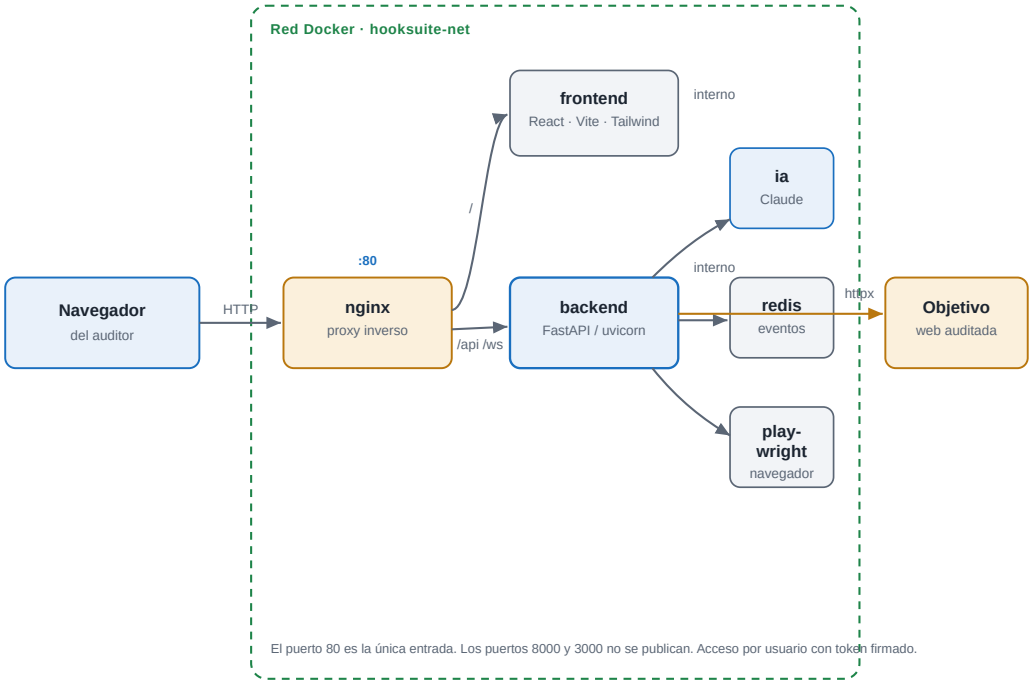


Figura 1: Arquitectura de contenedores de HookSuite. Nginx es la única entrada. El backend orquesta el rastreador, el banco de peticiones, la prueba masiva de valores, el módulo de IA y el navegador real, y sale al objetivo auditado. La autenticación no está en Nginx sino en el backend, que exige token en toda la API y en el canal en tiempo real (apartado 6).

Componentes

Componente	Tecnología	Rol	Puerto publicado
nginx	nginx:alpine	Proxy inverso y única puerta de entrada. Reparte entre frontend y backend	configurable, 80 por omisión
frontend	React, Vite y Tailwind (imagen propia)	Panel web servido como estático	ninguno
backend	FastAPI y uvicorn (imagen propia)	API de auditoría: proxy, rastreador, banco de peticiones, prueba masiva de valores, utilidades, vulnerabilidades y canal en tiempo real	ninguno
redis	redis:7-alpine	Bus de eventos y estado efímero entre servicios	ninguno
ia	Python con el SDK de Anthropic (imagen propia)	Clasificador de vulnerabilidades con Claude (RF-08)	ninguno

playwright	Python y Playwright (imagen propia)	Navegador real para ataques dirigidos (RF-10)	ninguno

Solo el proxy publica puerto. Los demás servicios se hablan por la red interna y no son alcanzables desde fuera.

Enrutado

Toda petición entra por Nginx, que reparte según la ruta pedida con la configuración de infra/nginx.conf.

- La raíz va al frontend, que sirve el panel de acceso del producto. Antes esa puerta iba protegida por una credencial única del servidor web, igual para todos. Ya no.
- Las rutas de la API y del canal en tiempo real van al backend, y ambas exigen token. Sin él responden 401.
- Las dos rutas del fichero de autoconfiguración de proxy se retiraron al jubilar ese modelo (apartado 3, RF-02), y también se retiró la ruta hacia el laboratorio vulnerable (apartado 6).

El reparto de responsabilidades cambió durante esta práctica, y conviene decirlo porque es una decisión de arquitectura y no un detalle. Antes Nginx era quien autenticaba, con una credencial única que no distinguía personas, y el backend no pedía nada. Ahora Nginx solo reparte, y quien autentica y autoriza es el backend, que es el único que puede saber de quién es cada dato.

Decisiones técnicas

Las cuatro de más peso, cada una con lo que se decidió, por qué y qué cuesta.

El bus interno entre el backend y el módulo de IA (RF-08)

- **Qué se decidió.** El panel pide la auditoría por la ruta normal, con su credencial. El backend publica la orden en un bus de eventos interno y el módulo de IA la recoge de ahí. Cada mensaje lleva el espacio de datos de quien pidió la auditoría, de modo que los hallazgos aterrizan en su sesión.
- **Por qué.** El control de acceso por usuario dejó fuera al propio módulo, que preguntaba al backend por HTTP y empezó a recibir un rechazo por falta de credencial. Las otras dos salidas eran peores. Darle una credencial de servicio abre una excepción justo en la pieza cuyo valor es no tenerlas, y dejar esa ruta sin control permitiría a cualquiera encolar una auditoría contra el objetivo que quisiera desde nuestra infraestructura. El bus no necesita credencial porque la frontera la pone la red: no está publicado.
- **Qué cuesta.** Un control de acceso falla cerrando y un bus falla abriendo. Un mensaje que llegara sin dueño podría repartirse por si acaso, que es exactamente la fuga que cerraron las cuentas, así que un mensaje sin dueño se descarta. Hay pruebas de eso y de que dos auditores no ven lo del otro (apartado 7).

El Repeater se separa de la sesión compartida y de las redirecciones (RF-05)

- **Qué se decidió.** Las cuatro herramientas que lanzan peticiones comparten una sola función de envío, y esa función gana dos interruptores que solo el Repeater cambia: no seguir las redirecciones y no usar el cliente compartido.
- **Por qué.** Un Repeater que sigue las redirecciones nunca enseña el 302, porque muestra la última parada y pierde por el camino la cabecera Location y cualquier Set-Cookie del salto, que en auditoría es justo lo que interesa. Y el cliente compartido custodia las cookies del objetivo, de modo que dos envíos idénticos no daban el mismo resultado. La justificación completa está en RF-04, apartado 3.

- **Qué cuesta.** Nada a las otras tres herramientas, porque la librería de red permite decidir las dos cosas petición a petición. Se eligió eso antes que construir un segundo cliente, que habría duplicado la configuración de red y con ella la desactivación de la verificación de certificados, que conviene tener en un solo sitio.

Del mismo cambio salió un arreglo que beneficia a las cuatro herramientas. Las cabeceras que el producto muestra como enviadas se leen ahora de la petición que la propia respuesta conserva, y no de un diccionario construido aparte. En una herramienta de auditoría eso no es un detalle de presentación, porque la evidencia que se muestra es la que acaba copiada en un informe.

Del fichero de autoconfiguración de proxy a httpx (RF-02)

- **Qué se decidió.** El servidor ejecuta las peticiones directamente con httpx, en lugar de interceptar el tráfico del navegador del auditor.
- **Por qué.** Es la decisión de arquitectura de más peso heredada de la Práctica 1. Aquel modelo quedó expuesto y se saturó con tráfico de bots. El cambio reduce la superficie y simplifica el flujo.
- **Qué cuesta.** Se pierde la captura pasiva del tráfico del navegador, que es justo lo que retoman después DevTools (RF-09) y el navegador real (RF-10). Versión original, nueva y justificación en el apartado 3.

El frontend que se toma como base

- **Qué se decidió.** Tomar como base la interfaz que corría en producción, no la que estaba en el repositorio.
- **Por qué.** No eran la misma. La de producción se había reescrito directamente sobre el servidor y no existía en ninguna rama, así que lo primero fue devolverla al control de versiones. No es un cambio de tecnología: el informe de la Práctica 1 ya declaraba React, Vite y Tailwind.
- **Qué cuesta.** Hubo que reconstruir su historia para poder versionarla. El proceso se cuenta en el apartado 7.

SECCIÓN 5

Funcionalidades implementadas

Cada funcionalidad con lo que hace, cómo se usa y qué requisitos cubre. El panel tiene seis pestañas más la pantalla de acceso, y el módulo de IA trabaja por detrás, sin pestaña propia.

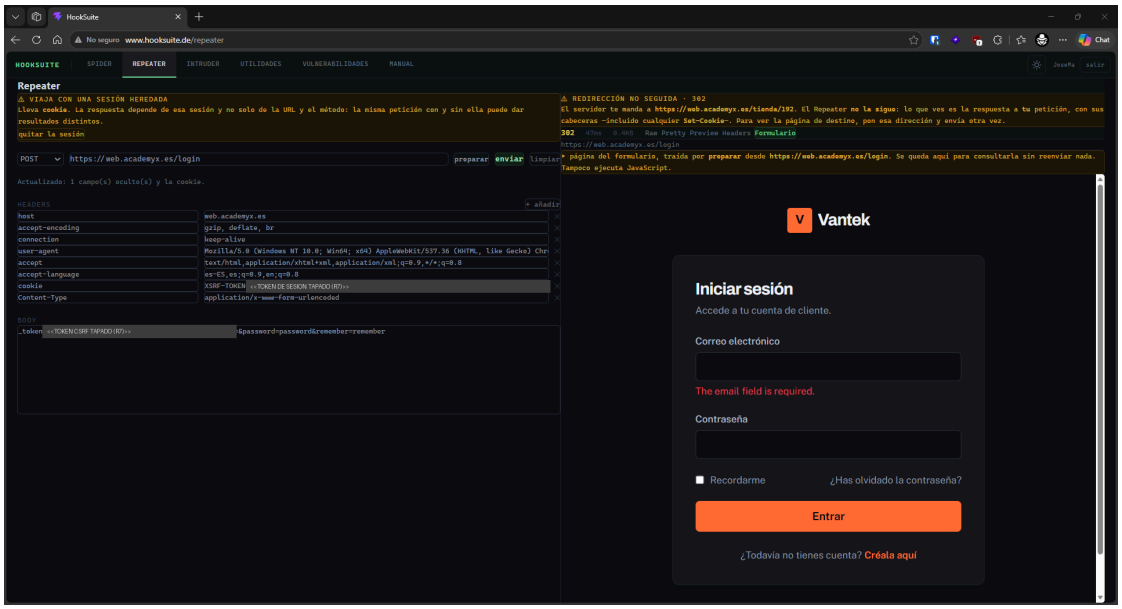


Figura 2: RF-05 — el Repeater tras pulsar «preparar» sobre un formulario traído del Spider, con cuatro cosas a la vez: el aviso de que la petición viaja con una sesión heredada y el botón que la retira. El acuse de «preparar», «1 campo(s) oculto(s) y la cookie» actualizados. El aviso de redirección no seguida con el 302 y la dirección a la que manda el servidor. Y la pestaña Formulario, que muestra la página pedida con sus estilos sin reenviar nada. Los valores de sesión van tapados.

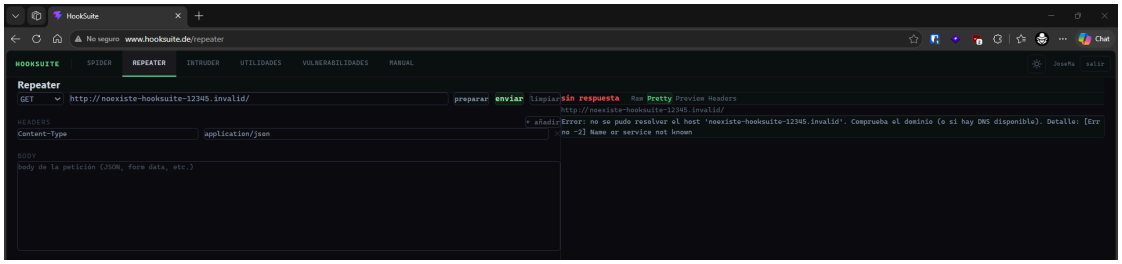


Figura 3: RF-05 — un fallo de red deja de disfrazarse de respuesta. Donde iría el código HTTP aparece «sin respuesta» en rojo, y el tiempo y el tamaño desaparecen de la barra: 0 ms y 0,0 KB no son medidas, son la ausencia de ellas. Debajo, la dirección a la que se envió y el motivo exacto, el host no se pudo resolver.

Las funcionalidades, una a una: qué hace, cómo se usa y qué requisito cubre

Lo anterior cuenta lo que cambió en cada fase. Esta tabla cuenta lo que hay, pantalla por pantalla, tal como queda en la versión congelada: el panel tiene seis pestañas más la pantalla de acceso, y la columna de la derecha dice dónde se ve cada una sin repetir aquí figuras que ya están en otros apartados.

Pantalla	Qué hace y cómo se usa	Requisitos	Dónde se ve

Spider	Rastrea el objetivo siguiendo enlaces y formularios y levanta su mapa. Se escribe la dirección, se lanza, y la tabla se va llenando. Se puede detener a medias, filtrar por método y mandar una petición al Repeater. Las cabeceras que muestra son las que se enviaron de verdad, no una plantilla	RF-03	figuras del apdo. 7 (antes y después del pulido)
Repeater	Banco de trabajo de una petición: se modifica y se reenvía exactamente lo que se ve en pantalla. Avisa si la petición lleva sesión heredada y ofrece quitarla. El botón «preparar» convierte la plantilla de un formulario en una petición que funciona. La respuesta se lee en Raw, Pretty, Preview, Headers y la página preparada	RF-04 · RF-05	cinco figuras en los apdos. 5 y 8
Intruder	Prueba muchos valores en un punto marcado de la petición y señala las respuestas que se salen del patrón. Se marca el campo, se elige diccionario, se lanza y se puede cancelar sin dejar el servidor trabajando	RF-06	figura en este apartado
Utilidades	Cuatro herramientas que no salen a la red: codificador (base64, URL, HTML y lectura de JWT), generador de huellas, probador de expresiones regulares y generador de payloads. Se pega el texto y se ve el resultado al lado	RF-07	figura en este apartado
Vulnerabilidades	Panel donde se dispara la auditoría con IA y donde aterriza su resultado. Lo que está visto funcionando es el camino degradado: lo que la IA no pudo analizar sale aparte y con su motivo. Nunca se ha visto un hallazgo analizado con su severidad, porque ninguna auditoría ha llegado a devolver uno. La pantalla queda sin pulir y sin terminar	RF-08 · RNF-06	figura del apdo. 7 (operación degradada)
Manual	Manual de uso dentro del propio producto, control por control y navegable por pastillas. No lo pide ningún requisito: salió del pulido, al comprobar que explicar una pantalla destapa sus defectos	— (mejora)	figura del apdo. 7
Acceso (fuera de la barra)	Alta con código de invitación y entrada con token firmado por el servidor. Cada cuenta ve solo sus propios datos	RF-11 · RF-12	cuatro figuras de producción en el apdo. 6

Tres piezas trabajan sin pestaña propia, y conviene decirlo porque quien busque su pantalla no la va a encontrar: el proxy del lado servidor (RF-02) es el que ejecuta las peticiones por debajo de Spider, Repeater e Intruder. El navegador real (RF-10) lo dirige el módulo de IA, no una persona. Y el **cortafuegos por sesión** (RNF-08) es un agente del sistema. La séptima pestaña que hubo, Red (RF-09), se retiró al descartar ese requisito, su figura se capturó antes de retirarla.

Tres notas de honestidad sobre esta tabla.

- **La pestaña que se llama Spider vive en la ruta /proxy**. Es un resto del nombre que tenía antes del pivote de RF-02, y se deja dicho porque cualquiera que mire las rutas lo va a notar.
- **El manual interno cubre dos de sus cinco secciones**: Spider y Repeater escritas, e Intruder, Utilidades y Vulnerabilidades marcadas como pendientes dentro del propio producto. Es la misma regla que esta memoria aplica a sus huecos: quien usa la herramienta ve que falta, en vez de encontrarse una sección vacía.
- **El pulido de esta fase llegó a dos pantallas, no a seis.** Spider y Repeater se repasaron enteros y el resultado está publicado, quince y dieciséis defectos, apartado 7. **Vulnerabilidades se empezó y no está terminada, y así se dice en su fila. E Intruder y Utilidades se entregan como estaban en la Práctica 1**: el único cambio del Intruder es la corrección de su cancelación, y el de Utilidades es documental, la lectura de JWT ya existía, y lo que estaba

equivocado era la nota del 29 de septiembre que la daba por ausente. Conviene decirlo porque la tabla de arriba describe **lo que cada pantalla hace**, no cuánto se ha repasado.

Y una corrección de lo que esta memoria dio por roto. La verificación del 29 de septiembre anotó que «el codificador no soporta JWT aunque RF-07 lo promete». Hoy sí lo soporta: la lista de conversiones del codificador incluye la lectura de JWT, junto a base64, URL y HTML. RF-07 queda cumplido sin salvedades.

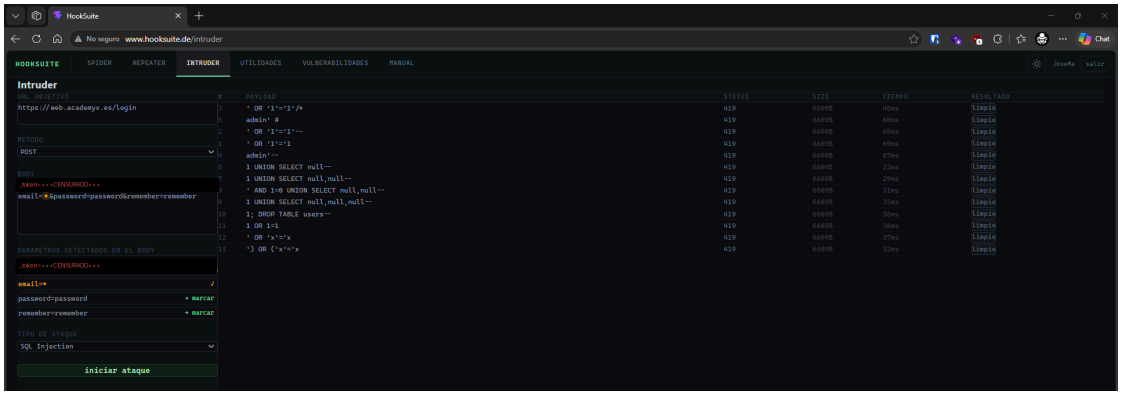


Figura 4: RF-06 — el Intruder con un ataque de inyección SQL lanzado contra el formulario de acceso del objetivo propio. El diccionario está a la vista: trece payloads numerados, de ' OR '1'='1 a 1. DROP TABLE users–, cada uno con su código, tamaño, tiempo y veredicto. A la izquierda, los parámetros que el Intruder detecta solo dentro del cuerpo de la petición, y cuál va marcado con el asterisco. Los trece responden 419 y el panel los marca limpio: el objetivo exige un testigo CSRF válido en cada envío y rechaza el reenvío, de modo que lo que la captura demuestra es el mecanismo de fuzzing, envío, medida y clasificación, y no un hallazgo. El testigo va tapado.

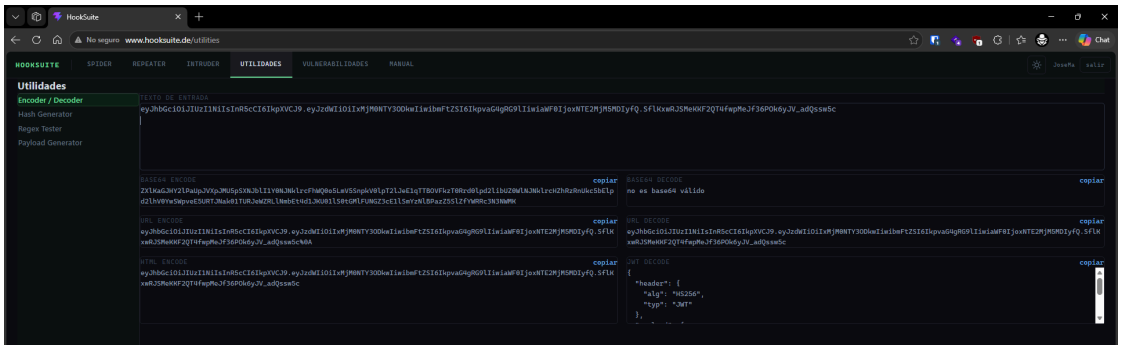


Figura 5: RF-07 — el codificador con un JWT en la entrada y sus cuatro conversiones resueltas a la vez: base64, URL, HTML y la lectura del JWT, que descompone la cabecera (HS256) y la carga útil. Es justo la conversión que la verificación del 29 de septiembre dio por ausente, y la que cierra RF-07 sin salvedades. El testigo mostrado es el ejemplo público de documentación, la carga útil es el «John Doe» de manual, no una credencial del producto.

Nada de lo que se entrega funciona con datos simulados

El enunciado lo pide con estas palabras: si una integración externa no está disponible y hay que simularla, **se indica de forma explícita en la memoria y en el vídeo**. Aquí no hace falta simular ninguna, y conviene decirlo con los interruptores delante porque el código sí trae los mecanismos para hacerlo.

- **La única integración externa del producto es la API del modelo**, y no se simula. Cuando falta la clave o la llamada falla, el producto no inventa un resultado: devuelve el estado degradado con su motivo (RNF-06, apartado 7). Es exactamente lo contrario de simular.
- **El navegador real se usa de verdad.** El interruptor MOCK_PLAYWRIGHT está en false en los dos servicios del fichero de composición y en el de ejemplo, así que la automatización va contra un navegador y no contra una respuesta escrita a mano.

- **El panel no sirve datos de ejemplo.** El interruptor USE_MOCKS está fijado a false en el código del frontend y ninguna variable de entorno lo enciende: haría falta editar el código y reconstruir.
- **Lo que sí hay que declarar, porque es verdad y es un defecto menor:** el módulo de datos de ejemplo sigue viajando dentro del paquete del frontend aunque nadie lo muestre, y menciona un laboratorio que ya salió del producto. No se ve en pantalla, pero está en lo entregado. Se retira después de la entrega.
- **El cliente simulado del modelo existe solo en las pruebas** (ia/tests/test_tres_estados.py), para poder demostrar RNF-06 sin gastar saldo, y se declara así en el apartado 7. No forma parte del producto.

SECCIÓN 6

Seguridad del producto

Modelo de amenazas, secretos, autenticación, validación de entradas, dependencias y datos personales. (Base en REQUISITOS.md apartado 3.) Contenido ya verificado en esta práctica:

Secretos: el incidente de las claves

En el historial de git había claves de API de Anthropic. Verificado el 4-oct: **las cuatro están muertas** (la API responde 401 / «API key is invalid.»). Aun así, el enunciado (apartado 7) obliga a limpiarlas del historial, no basta con borrarlas en un commit nuevo.

- El 4-oct se **reescribió el historial** con git-filter-repo: se redactaron las claves por patrón y se eliminaron los .env rastreados y la basura (node_modules, venv de Windows), conservando el código, las 7 ramas y el .env.example (con la clave ya redactada).
- **Ejecutado en GitHub el 4-oct** (force-push de las 7 ramas, con copia de seguridad previa). Verificado sobre un clon nuevo: **0 claves en todo el historial** de las ramas. El repositorio público queda sin secretos.
- Salvedad: GitHub conserva un tiempo los refs internos de los **pull requests** antiguos apuntando a los commits viejos. Los purga por su cuenta. Como las claves están muertas, no supone riesgo.

LIMPIEZA EJECUTADA Y VERIFICADA

Exposición de la caja: cómo estaba

Estado de partida, verificado en vivo el 29-sep y el 4-oct (evidencia fotográfica en el apartado 7):

- La API :8000 **no tenía autenticación**: lo declaraba su propia openapi.json (40 operaciones sin seguridad) y /health respondía sin credenciales.
- El Basic Auth del :80 se **evitaba** sirviendo el frontend directamente por :3000.
- El servidor **no tenía cortafuegos** (política de aceptación por defecto) ni protección contra fuerza bruta, y acumulaba unos **127.000 intentos de acceso en 7 días**.
- DVWA **no** estaba expuesto a internet (solo red interna), lo que corrige un supuesto previo nuestro.

Endurecimiento aplicado al inicio de la práctica (5-oct)

Cierre de la exposición

La API y el frontend dejaron de publicar puerto propio: **la única entrada es el proxy**. No bastaba con retirarlos del docker-compose.yml, porque el frontend llevaba la dirección del servidor incrustada en tiempo de compilación y hablaba directamente con la API. Hubo que **reconstruirlo para que use el mismo origen** y que sus llamadas (incluido el WebSocket) pasen por el proxy. Comprobado desde fuera tras el despliegue: los puertos directos **rechazan la conexión**, el proxy responde, y el paquete del frontend ya no contiene ninguna referencia a la dirección antigua.

Orígenes permitidos (CORS)

El backend aceptaba **cualquier** origen con credenciales, combinación además inválida por especificación: en la práctica, el servidor **reflejaba el origen de quien preguntara**, de modo

que cualquier web podía hacerle peticiones autenticadas. Se sustituyó por una **lista cerrada** configurable por entorno. Comprobado: un origen arbitrario ya no recibe permiso. Uno legítimo sí.

El laboratorio vulnerable: fuera del producto

El 5-oct se le retiró del proxy la ruta que lo publicaba, de modo que quedó accesible solo desde la red interna. El **13-oct**, al congelar, se retiró del producto **entero**: no está en el despliegue ni en la caja, ni el servicio, ni el contenedor, ni la imagen. Las pruebas de este proyecto se hacen contra una web propia del equipo, autorizada para ello, y mantener en el despliegue un sistema **deliberadamente vulnerable** que ya nadie usaba no tenía defensa. El único punto del código que todavía caía a él cuando llegaba una orden sin objetivo ahora **descarta esa orden y lo dice**, en lugar de auditar un host que ya no existe. **Lo que no se tocó, y se declara:** el módulo de navegador conserva una función de acceso a ese laboratorio y los datos de ejemplo del frontend lo siguen nombrando. Es código que no se ejecuta en el producto entregado, y reescribirlo el día de la congelación añadía riesgo sin ganar nada. Queda anotado en el apartado 9.

Endurecimiento del servidor

- **Cortafuegos** activo, con política de denegación por defecto y únicamente el acceso remoto y el proxy permitidos.
- **Bloqueo automático de fuerza bruta** contra el acceso remoto: en su primer minuto ya registraba 148 intentos fallidos y había bloqueado dos direcciones.
- **Acceso remoto solo por clave**: se desactivó la autenticación por contraseña. La prueba más limpia es el propio mensaje del servidor, que pasó de ofrecer «clave o contraseña» a admitir **solo clave**.
- **Limpieza de reglas abandonadas**: siete reglas heredadas de la P1, una de ellas con la dirección pública de un integrante y otra con un valor de ejemplo, se retiraron tras comprobar que ningún servicio escuchaba en esos puertos.
- **Agente de firewall duplicado**: se encontró un segundo proceso del agente, lanzado a mano en mayo y fuera del gestor de servicios, con permisos de administración del cortafuegos. Se terminó. Queda uno solo, gestionado por el sistema.
- **Sistema operativo**: se estrenó el núcleo ya instalado pero sin aplicar desde mayo. Antes de reiniciar se corrigió una condición de carrera en el arranque que habría dejado el agente del cortafuegos en bucle (ver apartado 7).

Dos matices honestos, para no atribuirnos de más

- **El acceso de administrador por contraseña ya estaba bloqueado** por la configuración por defecto del sistema: los 127.000 intentos registrados nunca tuvieron ninguna posibilidad. Lo que se cerró fue la autenticación por contraseña **a nivel general**. Se hizo igual porque deja de depender de un valor por defecto que una actualización podría cambiar, y porque cierra la puerta a cualquier usuario futuro.
- **El cortafuegos del sistema no filtra los puertos que publica Docker**, cuyas reglas se evalúan antes. Quien cerró de verdad la API y el frontend fue el despliegue, al dejar de publicarlos. El cortafuegos protege el resto del servidor, no esos puertos.

EXPOSICIÓN CERRADA Y SERVIDOR ENDURECIDO

Autenticación y aislamiento: de cuatro fugas a cero (8-oct)

Hasta que se rehízo el acceso, la API y el WebSocket **no tenían autenticación** por detrás del proxy: el único control era el Basic Auth de la entrada, que además no distinguía personas.

Eso no era una carencia, sino **cuatro fallos distintos**, los cuatro medidos en el entorno local de pruebas el 6-oct antes de tocar una línea de código. La tabla enfrenta cada uno con lo que lo cierra. La evidencia de los dos lados está en el apartado 7.

Cómo estaba (medido el 6-oct)	Cómo queda (acceso por usuario)
La API entera respondía sin credencial. La asimetría lo delataba: GET /health devolvía 401 por el Basic Auth de la entrada, pero GET /api/session/new devolvía 200 y entregaba un token. El proxy protegía unas rutas y dejaba /api abierta.	Un guardián aplicado **por router**, no ruta por ruta, para que una ruta nueva nazca protegida. Toda /api y /ws/ exigen token: sin él, 401 con cabecera WWW-Authenticate. El Basic Auth de la entrada se retiró, porque la credencial la pide ahora quien debe.
El token de sesión no era una credencial, era un nombre. El backend **creaba** sesión para cualquier cadena inventada, así que no había frontera que violar: bastaba escribir un token cualquiera en la URL.	El token lo **firma el servidor** y el espacio de datos se deduce del token, nunca de la URL ni del cuerpo. El navegador dejó de inventarse su identificador. La ruta que repartía identificadores sin valor de credencial se retiró.
La cookie de sesión de la web auditada se compartía entre todos. Se guardaba en una tabla global indexada solo por dominio, sin dueño. Un segundo usuario leía la cookie capturada por el primero, y se obtenía además sin token.	Las cookies viven dentro de la sesión de cada usuario y la tabla global desapareció. Verificado: el segundo usuario ya no obtiene la cookie del primero.
Los hallazgos se difundían a todos los paneles. Una vulnerabilidad publicada sin token aparecía en la sesión de dos auditores distintos: en una herramienta de auditoría, eso es una fuga de datos del cliente de otro.	Los dos endpoints que admitían peticiones sin token escriben ahora solo en el espacio de quien llama. Verificado: el segundo usuario no ve el hallazgo del primero.

Un quinto fallo, que introdujo la propia corrección

Se documenta porque el enunciado pide las pruebas que fallaron, y porque la lección vale más que el acierto. El guardián comprobaba que el espacio de datos nombrado **en la ruta** fuera el tuyo, pero **cinco rutas reciben ese nombre en el cuerpo de la petición**, y ahí no miraba nadie: un usuario autenticado podía escribir en el espacio de otro con solo ponerlo en el cuerpo. Es el mismo error que este documento le reprocha al código anterior, dejar que el cliente elija dónde escribe, un nivel más abajo.

La peor de las cinco fue el Repeater, y se escapó del primer inventario por buscarla con el nombre de modelo equivocado. Ahí el daño no era ensuciar el historial ajeno: el Repeater mantiene **un cliente HTTP persistente por usuario** que acumula las cookies del objetivo auditado, el mecanismo que da servicio a RF-04, de modo que nombrar a otro en el cuerpo reutilizaba **su sesión ya autenticada contra la web auditada**.

Se descartó la corrección evidente, que era validar ese campo contra el token: funciona, pero deja el dato en manos del cliente y obliga a acordarse en cada modelo nuevo. Las rutas toman el espacio del token y **descartan** lo que venga en el cuerpo: lo que no se lee no se puede falsear. Comprobado como ataque real (apartado 7).

Desplegado y verificado en producción (8-oct)

Hasta esta fecha todo estaba verificado solo en el entorno local, y el 7 de octubre se comprobó **en vivo y en la propia instalación pública** que las fugas no eran teóricas: la API respondía sin credencial a cualquiera. El despliegue se hizo el **8 de octubre a las 06:52 UTC**, en cuatro pasos con punto de retorno previo: se etiquetaron las imágenes en servicio y se copió la configuración, se mergeó el cambio a la rama principal, se trajo el código a la caja con git reset –hard y se generaron los secretos, y se reconstruyeron los contenedores. La verificación inmediata, con el sistema ya en marcha:

Qué se midió	Resultado
Entrada del panel (/)	200, carga **sin pedir Basic Auth**: la contraseña compartida está retirada
API (/api/auth/yo)	401, **exige token**. La fuga nº 1 de la columna izquierda, cerrada
Reinicios del backend	0 — prueba que los secretos llegaron: el código se niega a arrancar sin ellos

Contenedores recreados	backend, frontend y nginx. Los otros cuatro, intactos a propósito

El dato de los **cero reinicios** no es decorativo: el acceso por usuario se diseñó para **no arrancar** si faltan el secreto de firma o el código de invitación, en lugar de arrancar sin autenticación pareciendo correcto. Que el contenedor esté sirviendo sin un solo reinicio es la prueba de que esa comprobación se superó.

El flujo completo del acceso, capturado en la instalación pública el mismo día del despliegue:

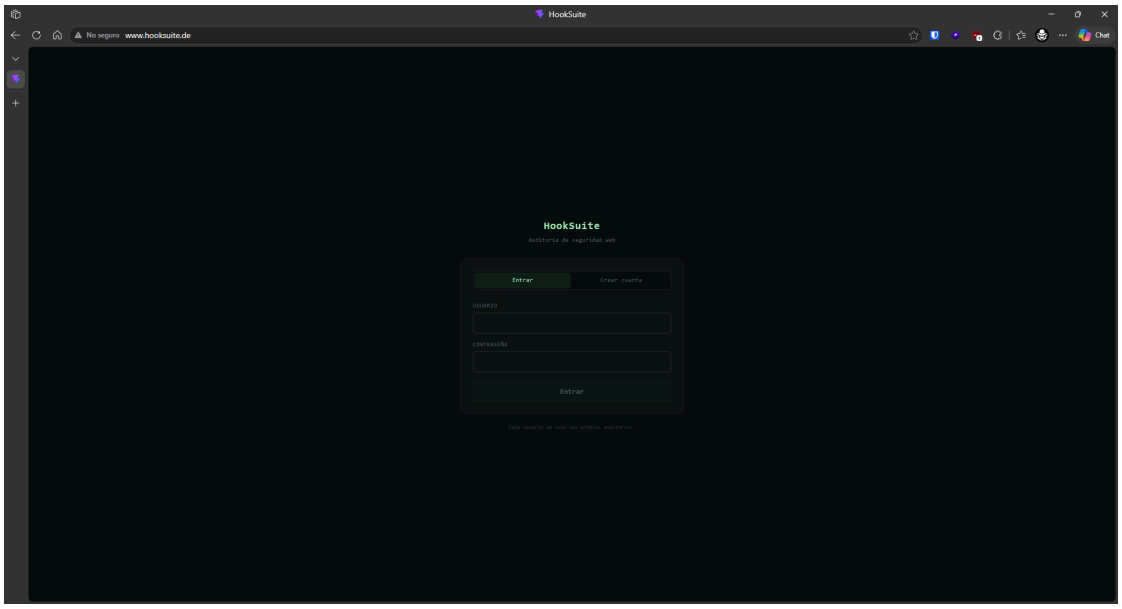


Figura 6: RF-12 — la entrada de www.hooksuite.de carga **sin** el cuadro de Basic Auth que antes pedía una contraseña compartida por todo el grupo. La puerta la guarda ahora el acceso por usuario.

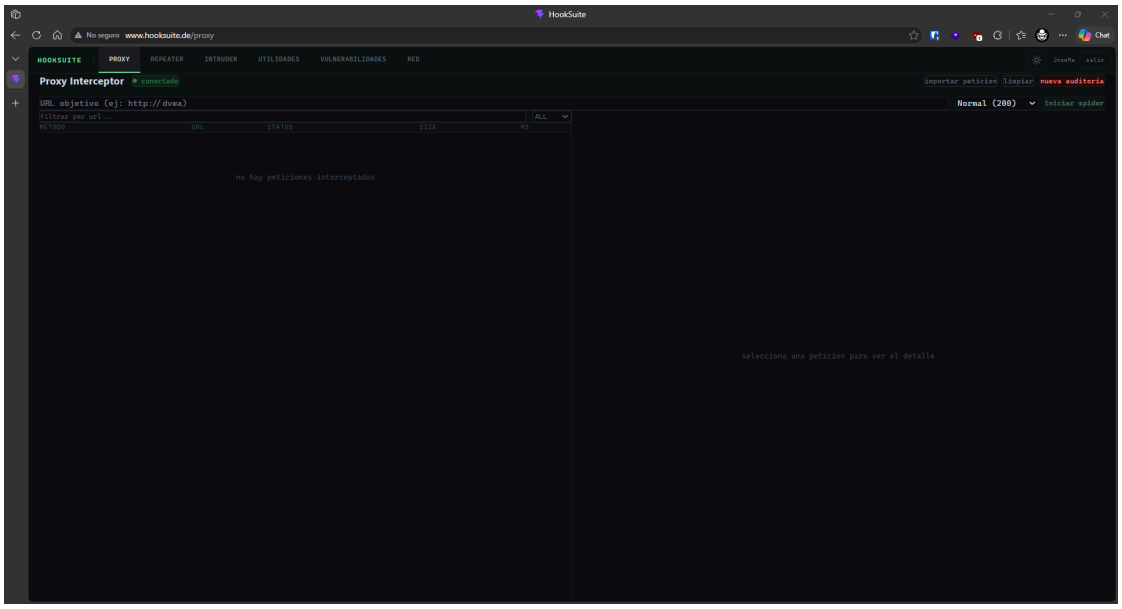


Figura 7: RF-12 y RNF-07 — sesión iniciada en producción, con el alta hecha mediante el código de invitación sobre una instalación que arrancó deliberadamente con cero usuarios: el usuario figura en la cabecera y el interceptor aparece como **conectado**, lo que prueba que el WebSocket viaja también autenticado y no solo la API.

Lo que no se pudo afirmar aquel día, y lo que quedó comprobado después. La primera comprobación por IPv6 devolvió 404: no porque el servicio no publicara por esa vía, sino porque el registro AAAA del dominio no apuntaba a la máquina. El 8 de octubre se comprobó contra la dirección IPv6 de la caja que el panel responde 200 y

que **los puertos cerrados el 5-oct siguen cerrados también por IPv6**, y el registro quedó corregido. La máquina de trabajo del grupo no tiene salida IPv6 (comprobado de nuevo el 15-oct), así que el recorrido dominio → IPv6 → panel está demostrado **por partes** y no de una pieza. Evidencia: docs/evidencias/cortafuegos-ipv6-caja-08oct.md.

LAS CUATRO FUGAS DE AISLAMIENTO, CERRADAS Y VERIFICADAS EN PRODUCCIÓN (8-oct, 06:52 UTC) · IPv6: REGISTRO CORREGIDO Y PUERTOS COMPROBADOS · FALTA UNIRLO EN UNA PETICIÓN DESDE UNA RED CON IPv6

Dos efectos del pulido del Repeater sobre la seguridad del producto (12-oct)

- **Cerrado: el último residuo del modelo anterior al login.** La petición de envío del Repeater llevaba en su cuerpo el identificador del espacio de datos, y el esquema del servidor seguía declarándolo. No se leía, el espacio sale siempre del token firmado, nunca del cuerpo, pero **sugería que el cliente puede elegir dónde escribe**, que es exactamente el hueco de autorización que cerró el acceso por usuario: con el nombre de otro en el cuerpo se reutilizaban **sus** cookies del login auditado. Se retiró del frontend el 12-oct a mediodía y del esquema del servidor esa misma noche. Un campo muerto que insinúa una capacidad que no existe es deuda de seguridad, no ruido.
- **Nuevo, y es una contrapartida asumida: el panel muestra ahora la cookie de sesión del objetivo.** Al corregir que las cabeceras mostradas sean las realmente enviadas (apartado 4), la Cookie del sitio auditado pasa a estar a la vista, como en la herramienta de referencia. Es lo correcto, la evidencia tiene que ser la petición que salió, y tiene un coste de método que conviene declarar: **cualquier captura de ese panel publica credenciales de un tercero**. Esas capturas van censuradas antes de entrar en esta memoria o en el artefacto. Ya ocurrió dos veces el mismo día. Lo que esto arrastra en el navegador del auditor, el borrador guarda esas cabeceras, se trata como limitación abierta en el apartado 9.

Lo que sigue sin cubrir

- **Sin TLS** (RNF-09): el acceso viaja en claro, así que el token de sesión es interceptable por quien esté en el camino. Pendiente de configurar en el proxy.
- **Credencial de servicio para los módulos internos.** Lo que este punto afirmaba se quedó atrás dos veces y se corrigió el 13-oct, y hoy son dos situaciones distintas. El **clasificador de IA ya no necesita credencial**: se le movió al bus interno precisamente para eso (apartado 4) y que RF-08 funcione de punta a punta es la prueba, porque si su camino de vuelta diera 401 el panel estaría vacío. **El agente de navegador sí sigue sin credencial, y ese es el hueco real**: sondea por HTTP las rutas protegidas sin cabecera de autorización y recibe 401. Queda como trabajo pendiente de RF-10, y la salida no es repartir una credencial más, es pasarlo al mismo bus que el clasificador.
- **Modelo STRIDE, validación de entradas, dependencias y datos personales**, abajo.

Modelo de amenazas (STRIDE)

El modelo se aplica a los seis componentes del apartado 4 y a la frontera que movió el acceso por usuario: hasta entonces autenticaba el proxy con una contraseña compartida. Hoy autentica el backend, por persona.

Una premisa que conviene decir antes de la tabla, porque un lector la echaría en falta: HookSuite es una herramienta **ofensiva**, y su función es enviar tráfico arbitrario a un objetivo que elige quien la usa. Por eso el control no puede ser «no permitir destinos arbitrarios», eso sería desactivar el producto, sino **que solo entren personas autenticadas y que cada una responda de lo suyo**. Lo que en otra aplicación sería una vulnerabilidad (hacer peticiones a donde diga el usuario) aquí es el requisito RF-02.

Amenaza	Dónde aplica en HookSuite	Qué lo contiene hoy	Qué queda abierto
S — Suplantación	Entrar como otro auditor. Hablar con la API sin ser nadie	Entrada única por el proxy. Las rutas /api/ y /ws/ exigen token firmado por el servidor, con el algoritmo **fijado en el servidor** y no leído del token. Alta con código de invitación. Contraseñas en hash	**Sin TLS, el token es interceptable en el camino** (RNF-09). El agente de navegador sondea sin credencial propia (arriba)
T — Manipulación	Tocar datos de otra cuenta. Manipular lo que recibe el agente de cortafuegos	El espacio de datos **sale siempre del token**, nunca de la ruta ni del cuerpo, es la lección del quinto fallo. Modelos declarados que rechazan la petición mal formada antes de la lógica. El canal del agente dejó de crearse con permisos para todo el sistema y valida lo que recibe	En claro y sin TLS, manipulable en tránsito
R — Repudio	Saber quién lanzó qué contra un objetivo	Cada petición del historial, cookie capturada y hallazgo cuelga de la cuenta que lo generó. El repositorio identifica autoría por confirmación	**No hay registro de auditoría propio**: los rastros son los del contenedor, que no se conservan ni se firman. Se declara, no se resuelve
I — Divulgación	Ver auditorías ajenas. Filtrar secretos o cookies del objetivo	Cuatro fugas de aislamiento cerradas y una quinta introducida y corregida. Las cookies del objetivo cuelgan de su dueño y no de una tabla global. Claves fuera del historial. Orígenes restringidos. El paquete del frontend ya no lleva la dirección directa de la API	Sin TLS todo viaja en claro. El borrador del Repeater guarda la cookie del objetivo en el navegador (apartado 9)
D — Denegación de servicio	Agotar el servidor, el saldo de la API o martillear al objetivo	Límite de peticiones simultáneas (RNF-05). **Techo de llamadas** a la IA, configurable y con degradación visible en el panel. El recolector de sesiones, que existía y no se ejecutaba, ahora corre. En el host, cortafuegos y bloqueo de fuerza bruta	La vía de huella digital no compara contra el techo. El Spider no tiene límite de ritmo propio contra el objetivo
E — Elevación de privilegio	Que un usuario del producto mande sobre el cortafuegos del host	El canal del agente pasó a tener permisos acotados y a validar. Se retiró un segundo agente lanzado a mano en mayo, fuera del gestor de servicios y con permisos de administración. Acceso remoto solo por clave	El agente de cortafuegos es, **por diseño**, un componente con privilegio sobre las reglas del host: su superficie es ese canal, y no desaparece

Validación de entradas

- **Modelos declarados en el borde, no comprobaciones dispersas.** El backend declara su entrada con modelos en seis ficheros, el de esquemas comunes y las rutas de alta, red, navegador, rastreo y vulnerabilidades, así que una petición mal formada se rechaza **antes** de llegar a la lógica.
- **Todas las rutas protegidas menos la puerta.** En main.py se registran nueve grupos de rutas y **todos** llevan la dependencia de protección salvo el de alta y entrada, que es por donde se consigue el token.
- **El identificador del espacio de datos no se acepta del cliente.** Se descartó la corrección evidente , validar el campo contra el token, porque deja el dato en manos de quien llama. Las rutas lo toman del token y punto. Es lo que cierra la familia entera del quinto fallo, no solo el caso encontrado.

- **Canonicalización en el Spider:** decodificar entidades, normalizar y deduplicar la cadena de consulta. Sin eso, el rastreo generaba direcciones infinitas y martilleaba al objetivo (la trampa de araña del 1-oct).
- **El importador de comandos curl** corrigió una expresión regular que recortaba el esquema y convertía direcciones seguras en inseguras.
- **En el módulo de IA, la validación es doble:** el esquema de respuesta se endureció tras **tres rechazos seguidos** de la API, validar contra una prueba propia no es validar, y el techo de llamadas corta por arriba, de modo que una entrada que dispare una auditoría enorme no se convierte en gasto ilimitado.
- **Lo que no se valida, y es deliberado:** la dirección del objetivo. Es la función del producto. El control está en el acceso, no en el destino.

Dependencias

Medido el **13-oct sobre el código congelado**, con las herramientas puestas para ello (npm audit sobre el fichero de bloqueo y pip-audit sobre los requisitos declarados). Se dan los números que dan las herramientas, sin redondear hacia abajo.

- **Frontend:** 241 dependencias resueltas (84 de producción, 113 de desarrollo, 45 opcionales) y **12 avisos** , nueve altos, uno medio, dos bajos. Restringido al árbol de producción, **ocho**: siete altos y uno bajo. Directas: el cliente HTTP del panel (alto), el enrutador (bajo, por su paquete interno) y la herramienta de construcción (alto, que entra en el árbol de producción a través del complemento de estilos).
- **Backend: 78 avisos en 12 paquetes** del árbol resuelto. Los que más pesan, y el primero importa más que los otros: **la librería que firma los tokens de sesión acumula trece avisos** en la versión fijada, es la pieza central del acceso por usuario, la de criptografía siete, la de formularios multiparte siete y el armazón ASGI bajo FastAPI otros siete.
- **El proxy de la P1 sigue pesando, y se comprobó antes de decirlo.** mitmproxy aporta tres avisos propios y arrastra cinco paquetes más (servidor web, compresión, serialización, protocolo binario y TLS). La tentación era declararlo código muerto y proponer quitarlo: **no lo es.** main.py y la ruta de alta importan el gestor que lo lanza como subproceso con su complemento, aunque RF-02 se implemente hoy con el cliente HTTP nuevo y las rutas del fichero de autoconfiguración se retirasen al rehacer el acceso. **Las dos arquitecturas conviven**, y esa convivencia es lo que mantiene viva esta superficie.
- **Política de fijado, desigual:** el backend fija sus doce versiones. El módulo de navegador tres de cuatro y el de herramientas de depuración las tres. **El módulo de IA solo una de cuatro**, así que su construcción no es reproducible al día de hoy. El frontend declara rangos, pero el fichero de bloqueo fija el árbol.
- **Qué no se ha hecho, y por qué: no se ha actualizado ninguna dependencia hoy.** Es el día de la congelación, el vídeo se graba sobre esta etiqueta y subir la librería de tokens o el cliente HTTP cambia el código que se entrega. Queda declarado aquí y en el apartado 9.
- **Honestidad de método:** pip-audit resuelve el árbol completo, así que esos 78 incluyen transitivas, y **no se ha medido cuáles son alcanzables** desde el camino de ejecución del producto. Decir «78 avisos» no es decir «78 agujeros explotables». Es decir cuántos hay que mirar.
- **Por qué los paquetes se nombran aquí por su papel y no por nombre y versión.** El repositorio es público y el producto está desplegado **sin TLS**: publicar la lista exacta de paquete, versión y número de avisos es publicar un mapa de por dónde entrar. Las versiones están en los ficheros de requisitos para quien tenga acceso al repositorio, y cualquiera puede reproducir la medida con las dos órdenes citadas arriba. Es la misma regla que se aplicó

a las claves: se cuenta el incidente, no se publica el valor. **Única excepción:** el proxy de la P1, que esta memoria ya nombra en los apartados 3 y 5 al explicar el pivote de RF-02. Ocultarlo aquí no taparía nada.

Tratamiento de datos personales

- **Qué guarda el producto, y de quién.** De cada auditor: su cuenta, nombre y contraseña **en hash**, nunca en claro, en un volumen propio, y colgando de ella su historial de peticiones, las cookies que captura del objetivo y sus hallazgos. Desde el cambio de acceso nada de eso se comparte entre cuentas.
- **De terceros, nada por iniciativa propia.** El producto no recolecta datos personales por su cuenta: lo que entra es lo que el auditor captura del objetivo que decide auditar. Las pruebas de esta práctica se hacen contra una **web propia del equipo y autorizada**, de modo que no hay datos de terceros en juego.
- **La cookie del objetivo es el dato más sensible que pasa por aquí**, porque equivale a una sesión abierta en la web auditada. Vive en el espacio de su dueño, y el editor avisa cuando una petición la lleva. Pero el **borrador** del Repeater la escribe en el almacenamiento del navegador: está declarado en el apartado 10 y no se tocó el día de la congelación.
- **Rastros a retirar antes de la entrega:** los usuarios de prueba creados en el entorno local de pruebas, y una regla heredada de la P1 que contenía la dirección pública de un integrante, esa ya se retiró con el endurecimiento del 5-oct.
- **Conservación y borrado:** no hay caducidad ni borrado automático de cuentas ni de historial. El recolector solo caduca el estado efímero de sesión. Se declara como lo que es: una decisión no tomada, no una medida.
- **Y el que lo atraviesa todo:** sin TLS, todo lo anterior viaja en claro (RNF-09). Es la limitación que más pesa en este apartado.

SECCIÓN 7

Pruebas y evidencias

Plan de pruebas, casos ejecutados, resultados y las pruebas que fallaron con su explicación. La matriz del apartado 8 relaciona cada caso con su requisito y su evidencia.

Dónde se tomó cada evidencia, porque cambia lo que vale. Salvo cuando se dice lo contrario, toda la verificación previa a cada despliegue se ejecutó en el **entorno local de pruebas**, una réplica del despliegue levantada con los mismos contenedores en una máquina del equipo, y se repitió después contra la instalación pública tras cada despliegue. Las dos se registran por separado y con su fecha: una prueba en el entorno local dice que el arreglo funciona. Solo la de producción dice que **está puesto**. Esa distinción no es formalismo: más abajo se cuenta un despliegue que habría pasado por correcto sin aplicar la configuración que se le pedía.

El plan de pruebas: los dieciocho casos que cita la matriz

El plan de trabajo original del equipo tenía **doce** casos, de modo que seis de los códigos que la matriz del apartado 8 cita (PR-03, PR-04, PR-10, PR-12, PR-15 y PR-20) no tenían caso escrito en ninguna parte: se redactan aquí por primera vez. Y los doce que existían llevaban resultados del 29 de septiembre, medidos sobre el prototipo antes de esta práctica, de modo que varios eran hoy literalmente falsos: PR-01 figuraba como fallo por un control de acceso que ya no existe, PR-16 decía que «toda la API responde sin credenciales», que es justo lo que se cerró, y PR-19 daba un error de servidor ya corregido. Copiarlos tal cual habría metido tres afirmaciones falsas en el entregable. Esos resultados tampoco son basura: son la foto del punto de partida, y este apartado pide expresamente las pruebas que fallaron con su explicación. Por eso la tabla lleva **dos resultados**, el del **29-sep** sobre el prototipo y el de **hoy** sobre el código congelado, y una última columna que dice **de dónde sale el resultado de hoy**, que es lo que permite juzgarlo: una reejecución, una figura de esta memoria, o nada, y entonces se dice.

ID	Req.	Qué prueba	29-sep (prototipo)	Hoy (código congelado)
PR-01	RF-01	Abrir la URL pública en un navegador limpio	401 Basic Auth: la credencial documentada no entra	**200**, sirve el panel · reejecutada en producción
PR-03	RF-03	Rastrear un objetivo y revisar el panel de detalle	no ejecutada	**pasa** · figura del apdo. 7: rótulos legibles y cabeceras reales
PR-04	RF-04	Enviar dos veces la misma petición y comparar	**sin caso escrito**	**pasa** · figuras del apdo. 7: mismo estado, tamaño y cuerpo. Distinto tiempo
PR-05	RF-05	Modificar y reenviar una petición. Ver la respuesta	envío correcto, historial vacío	**pasa** · figuras del apdo. 5. El historial se retiró por diseño (apdo. 9)
PR-06	RF-06	Marcar un campo y lanzar el Intruder	diccionarios servidos. Ejecución sin probar	**no reejecutada**: exige lanzar tráfico de ataque real, y su procedimiento del 29-sep nombraba el laboratorio retirado el 13-oct
PR-07	RF-07	Hash, codificadores y regex con casos conocidos	hash y codificadores correctos. JWT no soportado	**no reejecutada**: exige credencial de usuario
PR-08	RF-08	Publicar un hallazgo y recuperarlo por el panel	backend correcto. Faltaba el productor	**pasa** · recorrido completo verificado el 10-oct y figura del apdo. 7 (techo a 0 €)

PR-10	RF-10	Ejercitar el módulo de navegador	**sin caso escrito**	**no reejecutada**: hasta hoy el módulo ni siquiera estaba en la red interna. El despliegue del 13-oct lo corrige, pero no se ha ejercitado
PR-11	RF-11	Que cada usuario tenga su espacio de datos	tokens correctos por /api/session/new	**procedimiento caducado**: esa ruta se retiró al poner el acceso por usuario. Lo que hoy cumple RF-11 se prueba con PR-12
PR-12	RF-12	Que un usuario no vea los datos de otro	**sin caso escrito**	**pasa** · par de figuras A/B de este apartado, verificado 11/11 en el entorno local
PR-13	RNF-03	Servicio en pie y API publicada en la caja	/health correcto. Esquema de la API **abierto**	**pasa y mejora**: /health 200 y el esquema **ya no se publica** · reejecutada
PR-14	RNF-07	Sin claves en lo que se publica	por ejecutar	**pasa** · cero ficheros seguidos por git con clave, con control positivo · reejecutada
PR-15	RNF-08	Cortafuegos por sesión	**sin caso escrito**	**no reejecutada**: el agente está operativo y endurecido (apdos. 6-7), pero no se ha vuelto a ejercitar
PR-16	RNF-07	Llamar a la API sin ninguna credencial	**toda la API respondía**	**pasa**: 401, y los puertos 8000 y 3000 rechazan conexión desde fuera · reejecutada
PR-17	RNF-02	Conexión en tiempo real con el servidor	enlace correcto, sin autenticar	**pasa** · el indicador «conectado» de las figuras lo prueba ya autenticado
PR-18	RNF-09	Responder por TLS	sin HTTPS	**sigue sin TLS** · reejecutada. RNF-09 es el único requisito declarado pendiente
PR-19	RF-02	Estado del proxy del lado servidor	**500**, error de servidor	**pasa**: 401, el extremo responde y exige credencial · reejecutada
PR-20	RNF-05	Límite de peticiones simultáneas	**sin caso escrito**	**no reejecutada**: el límite está implementado y citado en el apdo. 7, pero sin prueba propia

Cómo leer esta tabla, dicho sin adornos. De los dieciocho casos, **seis se han reejecutado** contra producción, **seis los demuestra una figura** de esta memoria, **cinco no se han reejecutado** y se dice por qué, y uno, PR-11, tiene el procedimiento caducado porque cita una ruta que el cambio de acceso retiró.

Un aviso de método que vale para quien repita estas medidas. El frontend es una aplicación de una sola página, y el servidor devuelve su portada a **cualquier ruta desconocida**: una dirección inventada responde 200 igual que una real. Por eso, en las filas reejecutadas, un 200 no se ha dado por bueno sin mirar el tipo de contenido, y es lo que evitó concluir que el esquema de la API seguía publicado cuando lo que respondía era la portada. Las salidas completas están en docs/evidencias/pruebas-reejecutadas-13oct.md.

Evidencia de exposición (RNF-07 / PR-16), capturada el 4-oct

La prueba PR-16, llamar a la API sin credenciales, y la verificación del frontend confirman que, antes de endurecer, el producto quedaba accesible sin autenticación. Las capturas se obtuvieron antes de cualquier arreglo, porque la evidencia de un fallo se captura antes de corregirlo: el propio arreglo la destruye. Además de lo que muestran las dos figuras, se comprobó que el frontend servido por el puerto 3000 cargaba sin pedir credencial, evitando por completo el control de la entrada principal, que la entrada principal sí pedía credencial (el control existía, pero se podía rodear por el puerto directo) y que la credencial documentada en el informe de la P1 era rechazada con 401: ya no abría nada.

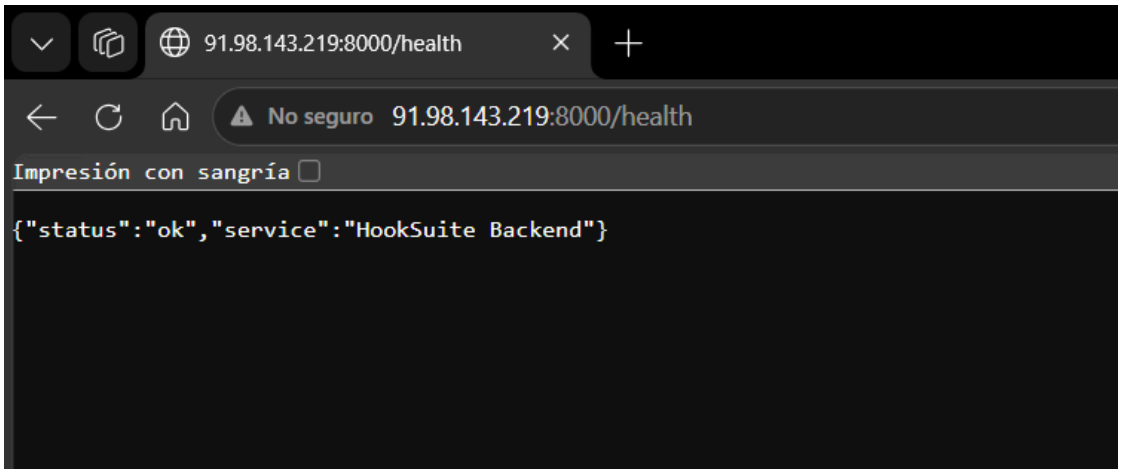


Figura 8: RNF-07 / PR-16 — GET /health de la API responde 200 sin credenciales ni cabecera WWW-Authenticate: la API no exige autenticación. La documentación interactiva (/docs) quedaba igual de abierta, exponiendo las 40 operaciones, incluidas las que lanzan tráfico contra terceros.

Los doce defectos del prototipo, corregidos y verificados

Todos se reprodujeron antes de corregirlos y se verificaron después en producción.

Qué fallaba	Causa	Requisito
El Spider entraba en bucle y martilleaba el objetivo	No canonicalizaba las direcciones: el mismo recurso generaba variantes infinitas	RF-03
El Spider podía dirigirse a servicios internos del propio servidor	Faltaba filtrar destinos no legítimos	RNF-07
El botón de detener el Spider no existía en la interfaz	El servidor aceptaba la orden, pero la pantalla no ofrecía forma de darla	RF-03
Detener el Spider no detenía nada	El bucle de rastreo no consultaba la señal de parada	RF-03
Un mismo formulario aparecía decenas de veces	No se agrupaban: se emitía uno por página visitada	RF-03
Cancelar el Intruder devolvía un error del servidor	La operación invocaba un método inexistente	RF-06
Dos operaciones del proxy devolvían error del servidor	Un error de sintaxis construía un tipo de dato equivocado	RF-02
Importar un comando curl rompía las direcciones seguras	Una expresión regular mal escapada recortaba el esquema	RF-05
Los avisos de error no decían qué había fallado	Se mostraba el texto crudo de la librería de red	RF-05
Dos pantallas no recibían avisos en tiempo real	El servidor guardaba una sola conexión por sesión y la segunda desplazaba a la primera	RNF-04
El agente del cortafuegos era manipulable por cualquiera	Su canal se creaba con permisos para todo el sistema y no validaba lo recibido	RNF-08
El estado de sesión podía crecer sin límite	El recolector existía pero nunca se ejecutaba, y cualquier identificador nuevo creaba una sesión	RNF-07

Defectos corregidos: antes y después (RF-03)

Un par de capturas tomadas sobre el producto desplegado, antes y después de corregir. El objetivo de las pruebas es una web propia del equipo, autorizada para ello.

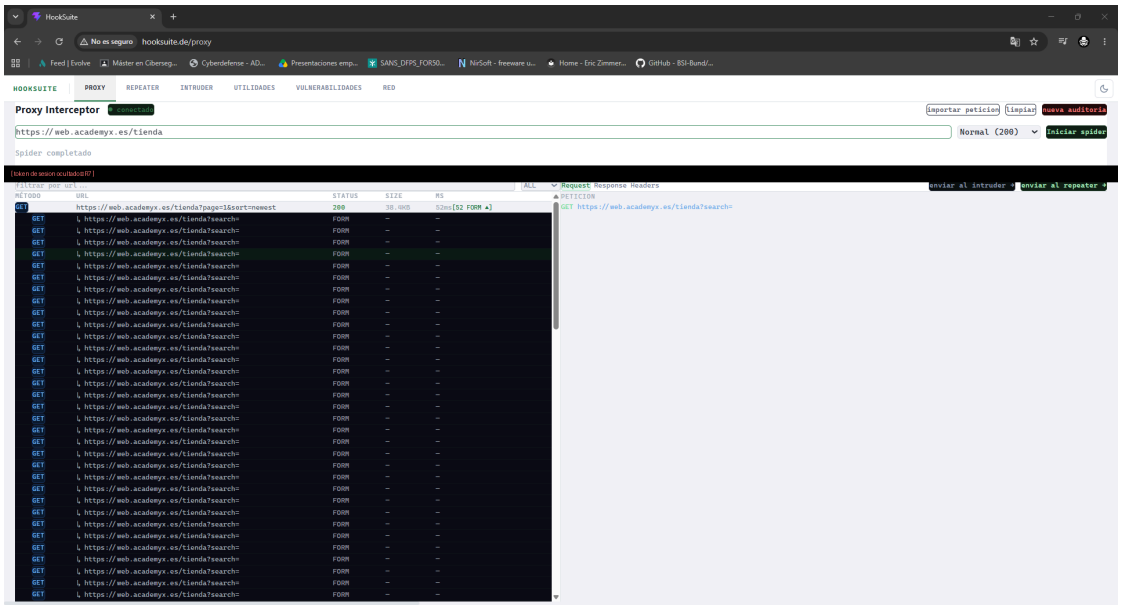


Figura 9: RF-03 — ANTES: una sola petición (/tienda?page=1&sort=newest) arrastra **52 entradas de formulario idénticas**, que inundan el panel de resultados.

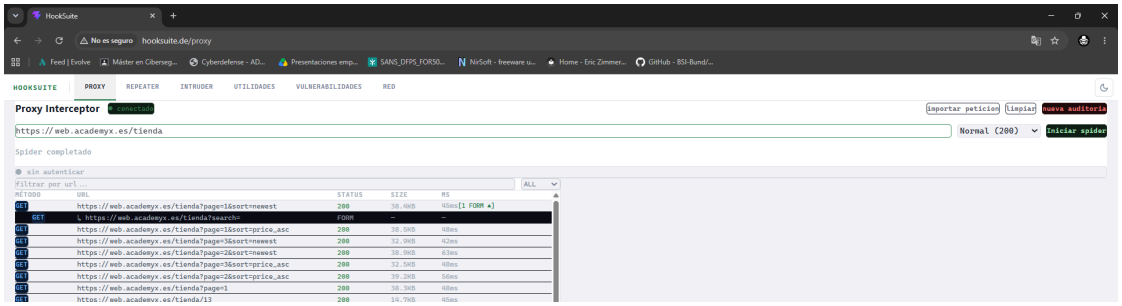


Figura 10: RF-03 — DESPUÉS: la misma petición de la misma página queda en **una sola entrada**. Solo cambia el contador: 52 → 1.

> **Nota de tratamiento de las capturas.** Las dos del estado anterior mostraban un identificador de sesión de la web de pruebas en la franja superior. Se publican con esa franja tapada, y los originales no salen del almacén interno. Las del estado posterior no lo contienen porque la sesión no estaba autenticada.

El pulido del Spider (11-oct): quince defectos, y catorce los encontró el uso

El 11-oct no se añadió funcionalidad: la jornada se dedicó a **pulir** lo que ya existía, pestaña por pestaña, empezando por la primera. De esa pasada salen quince defectos corregidos en el Spider. Lo que más dice de ellos no es el número, sino **cómo aparecieron**: catorce usando la herramienta y uno leyendo el código. Cada fila está atada a su entrada del diario de desarrollo, que es la fuente.

#	Qué pasaba	Cómo apareció	Diario
1	Pulsar «Detener» no paraba hasta terminar todos los formularios de la página en curso	usando	11:17
2	Tres textos de la pantalla describían algo distinto de lo que el código hacía	usando	11:17
3	El indicador de sesión decía «autenticado» sin que nadie se hubiera autenticado	usando	11:49
4	El mismo indicador fallaba en los dos sentidos: ni se encendía con sesión ni se apagaba sin ella	usando	12:45
5	El campo de URL se bloqueaba al iniciar, y no decía por qué ni cómo liberarlo	usando	12:51

6	Los tres botones de borrado hacían cosas distintas y nada permitía saber cuál usar	usando	13:01
7	La pestaña se llamaba «Proxy» y no es un proxy, sino un rastreador	usando	14:19
8	Los formularios colgaban de páginas ajenas: la pertenencia se adivinaba por prefijo de URL	usando	14:19
9	La columna de estado mezclaba códigos HTTP con la palabra FORM, que no es un código	usando	14:27
10	Al terminar, el rastreo no decía cómo había terminado	usando	17:52
11	El objetivo fijado se perdía al refrescar la página, y permitía lanzar otro encima	usando	17:52
12	El recuento del mensaje final no cuadraba con las filas del panel	usando	18:12
13	Las primeras filas capturadas no se alcanzaban con el scroll	usando	18:15
14	La etiqueta del desplegable de formularios insinuaba más de lo que había	usando	18:15
15	Los filtros escondían los formularios. Filtrar por POST vaciaba la tabla entera	leyendo el código	19:22

El patrón que se repite, y es el hallazgo de la jornada. En once de los quince, el backend **ya sabía** la respuesta y no llegaba a la pantalla: de qué página salía cada formulario, cómo había terminado el rastreo, cuál era el objetivo. No eran fallos de cálculo sino de **traslado**: funciones correctas que la interfaz no dejaba ver, y las pruebas que miran el backend no detectan nada de esto. El decimoquinto apareció al documentar el filtro para el manual: filtraba las filas **antes** de agruparlas, de modo que filtrar por POST, justo lo que se hace para ver formularios, vaciaba la tabla entera. Y dos de los quince los introdujimos nosotros al arreglar otros: están contados en la tabla y se detallan en el apartado 11, porque el recuento honesto incluye los defectos propios, no solo los heredados.

El pulido del Repeater (12-oct): dieciséis defectos, y una función retirada el mismo día

El 12-oct repitió el método del día anterior sobre la segunda pestaña: no se añadió alcance, se pulió lo que había. El resultado se parece al del Spider en el número y se diferencia en algo que conviene contar entero, porque es lo más instructivo de la jornada: **una función se añadió por la mañana y se retiró por la tarde**.

Nº	Qué pasaba	Cómo apareció	Diario
1	Un fallo de red se mostraba como una respuesta más, con el código HTTP 0 y medidas de «0 ms» y «0,0 KB»	leyendo el código	10:50
2	El panel no decía a qué dirección se había enviado: con varias peticiones seguidas no se sabía de cuál era la respuesta	usando	10:50
3	El cuerpo de la petición de envío mandaba el identificador de espacio de datos, residuo del modelo anterior al login	leyendo el código	10:50
4	«Limpiar» no era definitivo: al volver a la pestaña reaparecían la petición y sus cabeceras heredadas	usando	12:37
5	El Spider guardaba y enseñaba como enviadas unas cabeceras **fabricadas aparte**: otro User-Agent, un Host que nadie puso y ninguna Cookie	tirando de un hilo	12:50
6	El arreglo anterior seguía sin mostrar la cookie de sesión, porque la pone el cliente de red por debajo y no pasaba por ese diccionario	captura de producto	13:35
7	Los cinco rótulos del panel de detalle no se distinguían del contenido: se leían las cabeceras de respuesta como si fueran de petición	usando	13:30
8	Las respuestas se cruzaban: se veía una dirección con el cuerpo de otra petición, y no era reproducible a voluntad	usando	16:49
9	No había forma de probar la misma petición **sin** sesión: el único botón vaciaba la petición entera	usando	16:49
10	El Repeater seguía las redirecciones siempre, así que **nunca** se veía el 302 ni su Location	comparando con la herramienta de referencia	18:03
11	Dos envíos idénticos daban resultados distintos: el cliente compartido ponía cookies que no estaban en la petición	usando	18:37

12	Los formularios descubiertos por el Spider viajaban sin sesión, y todos los POST devolvían 419	usando	18:45
13	La petición que el Spider manda al Repeater nace vieja: es una plantilla con el token del rastreo, no una petición capturada	usando	20:24
14	«Limpiar» no borraba la página preparada, que se quedaba a la vista	usando	20:43
15	Dos avisos que ya no podían aparecer nunca: quedaba el código que los lee, no lo que los provoca	escribiendo el manual	21:03 y 21:24
16	El manual del Spider afirmaba de las cabeceras lo que el código no cumplía, y no documentaba un botón nuevo	escribiendo el manual	21:13

La función que no llegó a la noche, y por qué se cuenta en vez de omitirse. A las 10:08 el Repeater ganó un **historial** de peticiones y a las 17:28 se retiró entero: todo el comportamiento errático de la jornada giraba a su alrededor (respuestas cruzadas, entradas con la petición y la respuesta mal emparejadas) y tres arreglos sucesivos no lo dejaron estable, con la congelación al día siguiente. Un Repeater que mezcla respuestas es peor para un informe que un Repeater sin historial. Lo que ocupó su lugar es la decisión de diseño del apartado 9: **una petición cada vez**, con el estado «ocupado» visible desde la otra pantalla. La causa de fondo apareció cuatro horas después de retirarlo: el 302 invisible hacía que, al enviar un formulario que rebota, la respuesta del GET posterior pareciera la del POST. Se deja escrito porque ese es el orden real de los hechos: **primero se retiró el síntoma y después se encontró la causa**.

El patrón de los dieciséis, comparado con el del Spider. En el Spider, once de quince eran fallos de **traslado**: el servidor sabía la respuesta y la pantalla no la enseñaba. Aquí el patrón es otro y más incómodo: **siete de los dieciséis son fallos de fidelidad**, el producto mostraba algo distinto de lo que hacía. Cabeceras que no eran las enviadas, un 302 que no se veía, dos envíos iguales con distinto resultado, un fallo de red disfrazado de respuesta. En una herramienta de auditoría ese es el defecto más caro, porque **lo que la herramienta muestra es la evidencia que acaba en el informe**: un auditor que copie esas cabeceras documenta algo que no ocurrió.

El proyecto tenía un verificador configurado y sin usar. La regresión más tonta de la jornada (el botón de enviar dejó de hacer nada porque un borrado se llevó la declaración de una variable) la habría cazado el analizador estático del propio repositorio, que **nunca se había ejecutado**. Desde entonces va por delante de cada verificación y hoy no queda ninguna variable no declarada en el frontend. Lo que destapa de paso va al apartado 9: el frontend acumula veintitrés errores y dos avisos de otras reglas y **no pasa su propio analizador**. Los defectos que el asistente introdujo ese día, y por qué sus verificaciones no los cazaban, están en el apartado 11.

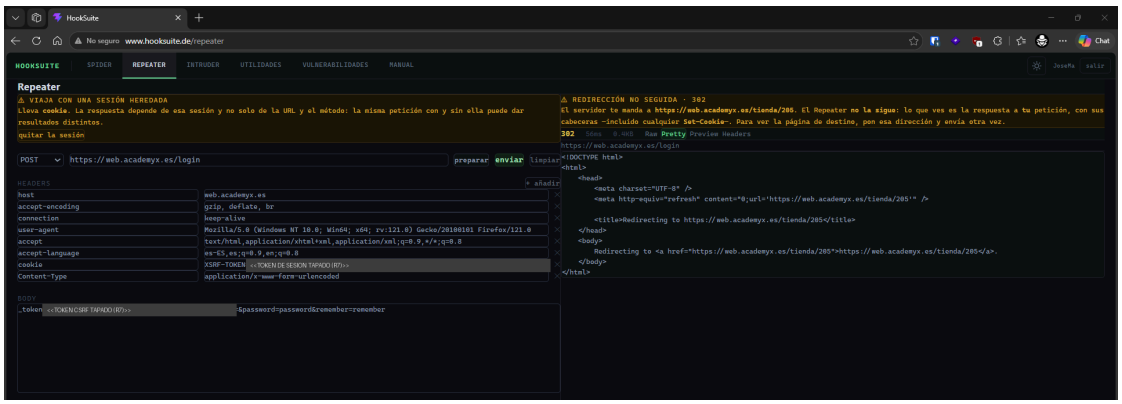


Figura 11: RF-04 — prueba de aceptación del Repeater con cliente propio, **primer envío**: 302, 0,4 KB y destino / tienda/205.

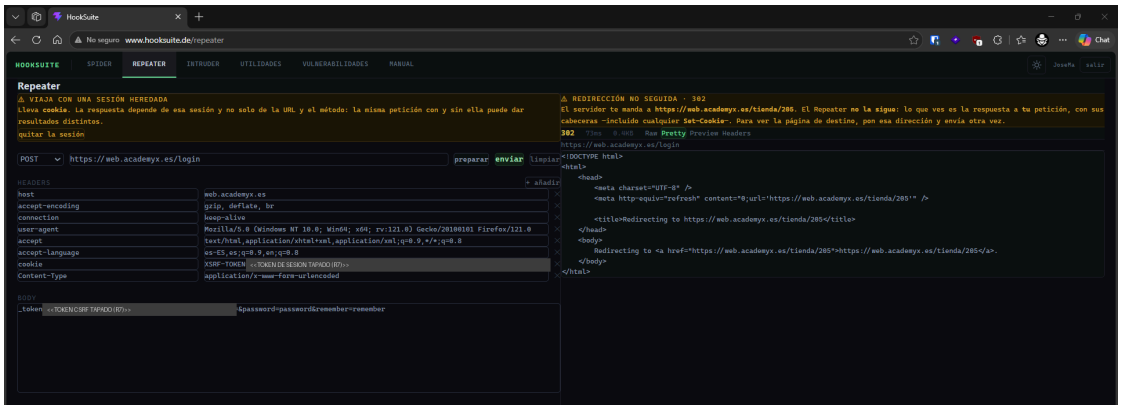


Figura 12: RF-04 — **segundo envío de la misma petición, sin tocar nada**: idénticos el estado, el tamaño, el destino y el cuerpo de la respuesta. Lo único que cambia es el tiempo (56 ms → 73 ms), y ese cambio es justamente lo que demuestra que son dos envíos distintos y no la misma figura repetida. Con el cliente compartido anterior, dos envíos iguales podían no coincidir.

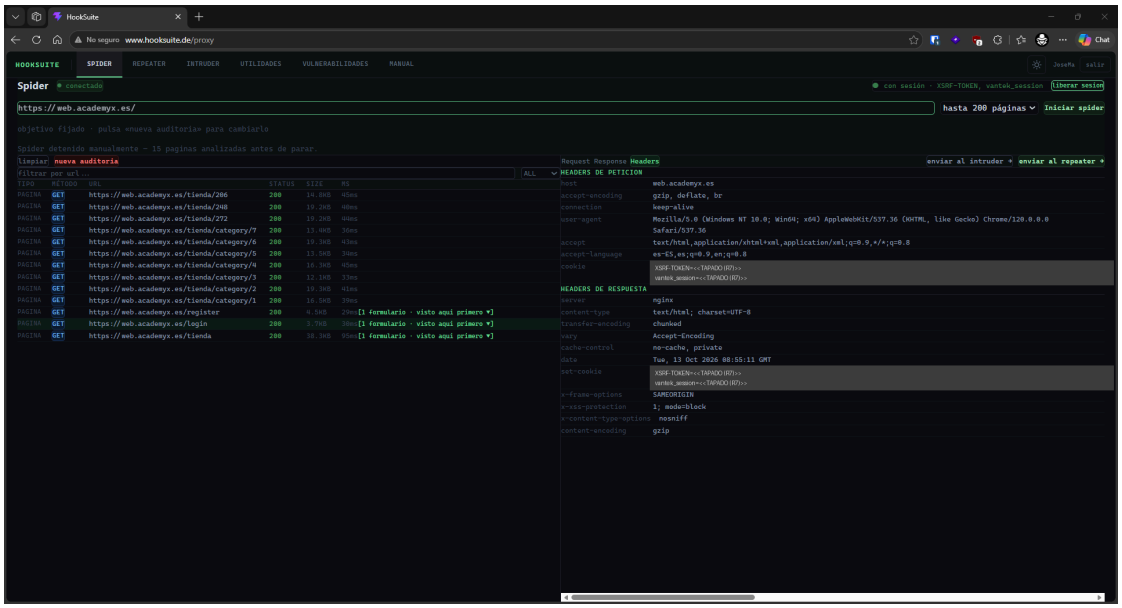


Figura 13: RF-03 — el panel de detalle después del pulido, con los dos defectos de fidelidad corregidos: los rótulos de sección se distinguen del contenido (antes compartían el mismo gris, y eso llevó a leer las cabeceras de respuesta como si fueran de petición) y las cabeceras que se muestran son **las realmente enviadas**, con la Cookie de sesión incluida. Los cuatro valores de sesión van tapados.

> **Nota de tratamiento de estas figuras.** Todas se tomaron en producción, sobre el código congelado, contra una web propia del equipo autorizada para ello. Desde el 12-oct el panel de cabeceras muestra la cookie de sesión del objetivo, es lo correcto, porque es lo que se envía, así que cada figura se publica con esos valores tapados y los originales no salen del almacén interno.

Condición de carrera detectada antes de reiniciar el servidor

El agente del cortafuegos y el servicio de contenedores no tenían orden de arranque entre sí. Si el segundo ganaba la carrera, **creaba una carpeta donde debía ir el canal de comunicación del agente**, comportamiento normal al montar un fichero que no existe, y el agente quedaba en un bucle de reinicios. El servidor llevaba sin reiniciarse desde mayo, así que nunca se había puesto a prueba. Se corrigió fijando el orden de arranque antes de reiniciar. Tras el reinicio, el contador de reinicios del agente marcaba **cero**: la carrera no llegó a producirse.

Otras evidencias ya en mano

- **Incidente de las claves y limpieza del historial:** ensayo de reescritura verificado antes/después (3 → 0 claves), detallado en el apartado 6.
- **Salidas de terminal** (health sin login, openapi.json sin seguridad, cierre de puertos, mensajes de error) guardadas como texto en el repo, no como foto (así ninguna captura publica por error un secreto).

El aislamiento (RF-12): las fugas capturadas antes, y la verificación después

Antes del arreglo, cuatro pruebas lanzadas el 6-oct contra el entorno local a través del proxy, como las haría un cliente, porque el propio arreglo borra esta evidencia y el apartado pide las pruebas que fallaron. Las cuatro confirmaron el fallo: la API responde 200 sin credencial mientras /health responde 401. Un token inventado obtiene sesión propia. Un segundo usuario lee la cookie de sesión capturada por el primero. Y una vulnerabilidad publicada sin token aparece en la sesión de dos auditores distintos. La captura íntegra, con la causa en fichero y línea de cada una, está en evidencias/fugas-aislamiento-06oct.md, con los valores de cookie inventados a propósito.

Después, con el sistema de acceso puesto, se repitieron las mismas pruebas. **Once comprobaciones, once en verde**: alta de dos usuarios distintos. Entrada de ambos. Contraseña incorrecta rechazada con 401 y no con un error del servidor. La API respondiendo con token y 403 al intentar tocar el espacio de otro con token propio, es decir, autenticación **y** autorización, que son cosas distintas. El segundo usuario sin ver ni el hallazgo ni la cookie del primero. Se probó además **como ataque**: un usuario autenticado intentó dirigir el Spider al espacio de otro nombrándolo en el cuerpo de la petición, el quinto fallo del apartado 6, y el rastreo fue a parar al espacio del atacante. Ni el código de invitación ni las contraseñas aparecen en ninguna salida: el guion de pruebas los mantiene en variables.

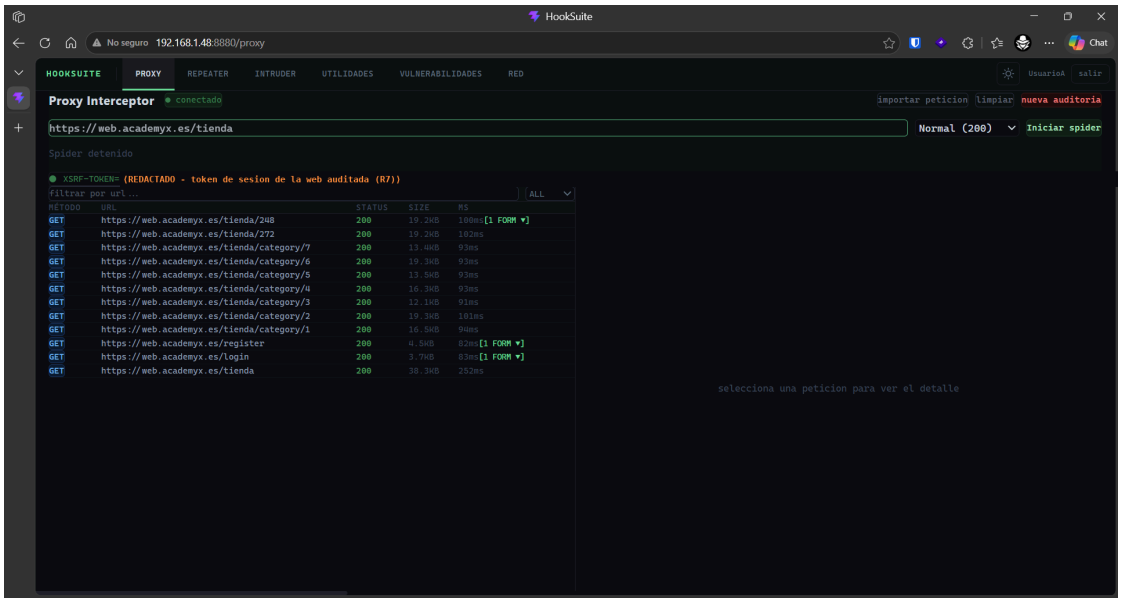


Figura 14: RF-12 — antes de rehacer el acceso este panel era común a todos. Ahora el usuario A, autenticado, ve las trece peticiones que su propio Spider capturó del objetivo autorizado. Compárese con la captura siguiente, tomada al mismo tiempo.

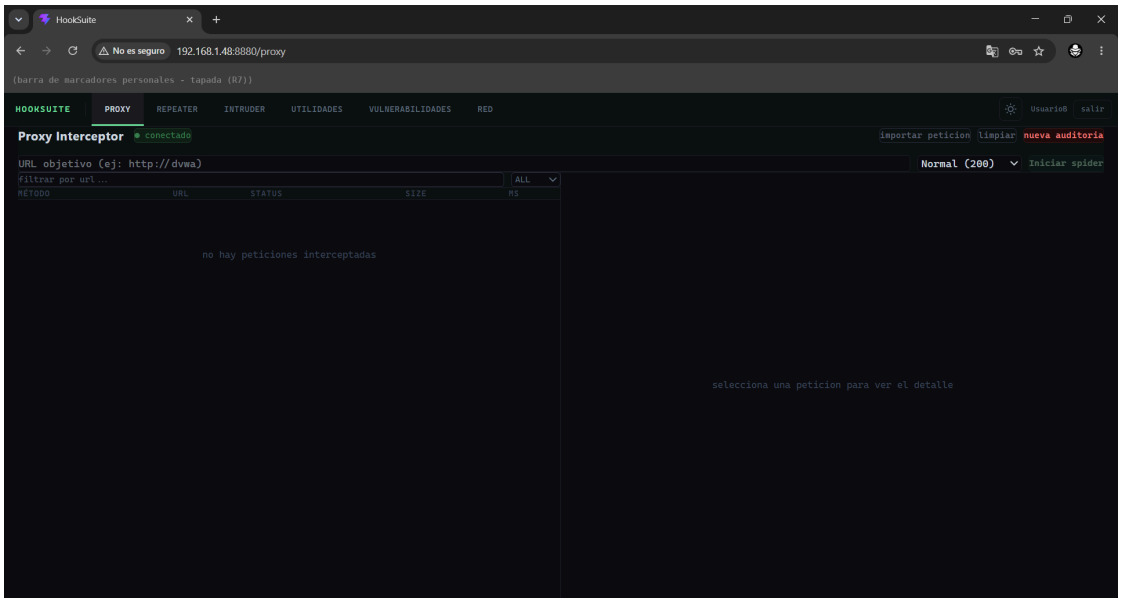


Figura 15: RF-12 — el contraste: el usuario B, autenticado a la vez y en otro navegador, ve su panel **vacío**. Antes del cambio habría visto exactamente lo mismo que el usuario A. Este par es la prueba del aislamiento por usuario. El alta de ambos exige el **código de invitación**: el registro no es abierto a propósito, porque la herramienta lanza tráfico contra terceros y con altas anónimas cualquiera podría atacar desde la infraestructura del grupo.

> **Nota de tratamiento de las capturas.** La del usuario A mostraba el valor íntegro de la cabecera XSRF-TOKEN, que es el identificador de sesión capturado **de la web auditada**: justo el dato que este requisito existe para no compartir. Se publica con ese valor tapado y la etiqueta a la vista, para que se vea qué se redactó. El original no sale del almacén interno. La del usuario B lleva tapada la barra de marcadores del navegador, ajena al proyecto.

El despliegue habría terminado «en verde» sin aplicar la configuración del proxy

La comprobación previa al despliegue del sistema de acceso, hecha **antes** de tocar la instalación pública, encontró un defecto no en el producto, sino **en el propio plan de despliegue**. El plan mandaba reconstruir «solo el frontend, el backend y el proxy», pero el proxy **no se construye**: usa una imagen descargada y su bloque del fichero de composición no cambiaba ni un byte, de modo que Compose no lo habría recreado. Como el proxy lee su configuración una sola vez al arrancar, el despliegue habría acabado **sin errores y sin aplicar nada**: el panel de acceso nuevo habría seguido detrás de la contraseña compartida que venía a jubilar. Era además el mismo fallo que ya había ocurrido dos días antes, con el mismo servicio, y la razón de que se repitiera es instructiva: **nombrar un servicio en un comando da la impresión de cubrirlo**.

El 10 de octubre apareció el caso espejo, que completa la lección. Se recrearon el frontend, la API y el módulo de IA **y no el proxy**, porque su configuración no había cambiado. Resultado: **502 en todo**. Los contenedores recreados reciben direcciones nuevas y el proxy resuelve los nombres de sus destinos una sola vez, al arrancar: seguía llamando a direcciones que ya no existían. Las dos mitades tienen la misma raíz, **el estado del proxy se fija al arrancar**, y de ahí sale una regla operativa que no estaba escrita en ningún sitio y ahora sí: cuando se recrea cualquiera de los servicios a los que el proxy apunta, se recrea también el proxy, forzando su recreación explícitamente. Se descartó recargar la configuración en caliente, que también habría funcionado pero depende de un detalle del sistema de ficheros en lugar de ser incondicional.

La instalación desde cero, ejecutada: dos defectos que solo aparecen al seguir las instrucciones

RNF-03 pide un **despliegue reproducible**, y hasta el 14 de octubre lo que respaldaba ese requisito era que el servicio **estaba en pie** en la máquina de producción, no que alguien pudiera levantarlo desde cero. Son dos cosas distintas, y la segunda se probó siguiendo el README paso a paso, sobre un árbol de trabajo independiente y con su propio proyecto de contenedores, para que nada del entorno de desarrollo se colara en la prueba.

Salieron dos defectos, y ninguno se habría visto leyendo el documento. El primero: con el puerto 80 ocupado en la máquina, el arranque falla, «address already in use», y el aviso del README ofrecía **un único valor concreto** como alternativa, que en esa misma máquina también estaba ocupado. Seguir las instrucciones al pie de la letra producía el mismo error **dos veces**, y a partir de ahí el documento ya no acompañaba. El segundo es más fino, y es el que importa: ese fallo **solo tumba el proxy**, el único servicio que publica puerto, mientras los otros cinco arrancan con normalidad. Quien no lea la salida del arranque y pase al paso siguiente ve cinco servicios en marcha y puede creer que ha instalado. Es la misma familia que el caso de arriba: **algo que parece verde y no lo está**.

El arreglo fue del documento, no del producto: el aviso deja de mandar a un número concreto, se añade cómo comprobar si un puerto está libre antes de elegirlo, y se advierte de que la comprobación final tiene que mostrar los **seis** servicios. Con eso la instalación llegó hasta la pantalla de acceso y se completó el alta de una cuenta con el código de invitación que genera el propio despliegue. **Lo que esta prueba no es:** la comprobación que pide la lista del enunciado, que una persona ajena al grupo instale el producto siguiendo el README, porque la condujo quien lo había escrito. Esa sigue abierta.

Dos defectos de montaje que impedían operar

Los dos aparecieron al reconstruir el entorno desde cero, y ninguno daba error visible al arrancar.

- **El proxy dependía del laboratorio para arrancar.** El fichero infra/nginx.conf hacía del laboratorio vulnerable un requisito de arranque del proxy, que fallaba con «host not found in upstream» si aquel no estaba levantado. Se resolvió retirando su ruta del proxy, de modo que el laboratorio quedó solo en la red interna y Nginx dejó de depender de él.
- **El navegador real estaba fuera de la red interna.** En docker-compose.yml el servicio playwright no declaraba networks: hooksuite-net, así que quedaba aislado y no hablaba con el backend pese a tener configurada su dirección. Es la causa raíz de que RF-10 y RF-08 no llegaran a operar. Se corrigió con una línea en el fichero de composición y se verificó con docker compose config, que sitúa ahora los seis servicios en la misma red.

Un módulo que parecía sano y no podía hablar con nadie (RF-08)

La prueba más barata de esta práctica fue una petición con curl, y descubrió un defecto que llevaba cuatro días en producción sin que nadie lo notara: **el módulo de IA no podía comunicarse con el backend** desde que se puso el control de acceso. El módulo preguntaba al backend cada cinco segundos si había trabajo, esa ruta quedó protegida junto con las demás, correctamente, y el módulo no presenta credencial ninguna, de modo que recibía un rechazo en cada intento.

Lo grave no es el defecto, es por qué no se vio. El código comprobaba si la respuesta era un éxito, y un rechazo por credencial no provoca ningún error, simplemente no es un éxito:

el bucle volvía a dormir cinco segundos y lo intentaba otra vez, escribiendo en el registro que estaba esperando instrucciones. Un operador que mirara ese registro habría visto un módulo en marcha y a la espera. Estaba en marcha, y no podía recibir nada. Es el mismo patrón que RNF-06 vino a cerrar un nivel más arriba, **un fallo que no se distingue de la normalidad**, y la razón por la que ahora el módulo falla ruidosamente: comprueba el bus al arrancar y se detiene si no está. Se midió con tres peticiones sin credencial contra la instalación de pruebas, las tres rechazadas. Evidencia como texto en docs/evidencias/canal-ia-backend-muerto-10oct.md.

Las dos contrapartidas del bus, probadas antes de confiar en él

Reconectar el módulo por un bus interno (apartado 4) cambia la forma del riesgo: un control de acceso falla cerrando, un bus falla **abriendo**. Así que lo que se prueba no es solo que el reparto funcione, sino que **no se puede repartir mal**. Dieciocho casos en dos conjuntos, con dobles para el bus, el gestor de sesiones y el orquestador: ni red, ni credencial de modelo, ni contenedores, cero coste.

Los dos que justifican el esfuerzo: **un hallazgo sin dueño se descarta**, no se guarda ni se emite (sin esa regla, un mensaje mal formado reabriría exactamente la fuga de aislamiento ya cerrada), y **dos auditores no ven lo del otro**, comprobado repartiendo a dos espacios y verificando que cada uno recibe solo lo suyo. Tres más cubren el fallo silencioso en el otro extremo: si la auditoría se interrumpe se publica el motivo, si nadie escucha el bus publicar lo dice en vez de dar el trabajo por encolado, y si una orden llega sin dueño no se audita. Reproducible con python3 backend/tests/test_bus_reparto.py y python3 ia/tests/test_bus_publicacion.py.

El recorrido completo del disparador, verificado de punta a punta (RF-08)

Durante cuatro días el disparador estuvo **construido y sin poder demostrarse**: el botón existía, el bus estaba cableado, y aun así una pulsación desde el panel no movía ni una línea del módulo de IA. El apartado llegó a listar tres hipótesis sin poder decidir entre ellas.

El 10-oct se cerró, y la forma de cerrarlo es parte del resultado: en lugar de seguir estimando sobre ventanas de tiempo aproximadas, **se anotó la hora exacta de la pulsación** y se midió contra ella. A partir de las 13:58:10 el servidor registra **una** petición al disparador, con respuesta correcta, y el registro del módulo de IA pasa de 192 a 376 líneas. Panel → autenticación → bus → módulo suscrito: el recorrido entero, por primera vez.

Lo que no se da por explicado. Cincuenta y ocho minutos antes, la misma pulsación no había movido nada, y entre una y otra **no se tocó el producto**, ningún fichero ni contenedor cambió. La explicación cómoda sería una sesión caducada, pero no encaja: el sondeo de instrucciones sigue devolviendo **no autorizado** ahora, con sesión válida y el disparador funcionando. Queda abierto y declarado, porque un apartado de pruebas que explica lo que no sabe explicar vale menos que uno que lo admite.

El techo de gasto, ejecutado: una auditoría entera por 0 € (RF-08)

Este caso estuvo declarado durante semanas como **preparado y no ejecutado**, y se deja constancia de que ya no lo está. Una auditoría del módulo de IA no tiene límite de consultas al modelo: el recorrido anida páginas × tipos de prueba × cargas, y cada carga es una consulta. Con el disparador funcionando, cada pulsación arranca ese recorrido completo.

El caso estaba diseñado para costar **cero**: fijando el techo en 0 por variable de entorno, ninguna consulta debe salir y la auditoría debe quedar marcada como truncada. **Ejecutado el 10-oct desde el panel, con sesión real.** La auditoría recorrió cuarenta y ocho elementos,

no salió **ni una sola** consulta al modelo, coste 0 €, y los cuarenta y ocho quedaron marcados como no analizados, cada uno con el motivo del techo. El techo no se limita a cortar: **el corte se ve**, que era la condición que se le puso al diseñarlo.

La mitad que falta del caso, y se declara: el techo en 1, donde debe pasar una sola consulta y el resto quedar como «no analizado». Esa sí necesita clave de API válida, porque la consulta que pasa tiene que salir de verdad.

⚠️ **HUECO** — responsable: **el grupo**. Ejecutar la segunda mitad del caso del techo, techo en 1, en cuanto haya clave de API válida: debe pasar una sola consulta y el resto quedar como «no analizado» con el motivo del techo. La primera mitad (techo 0) ya está ejecutada y capturada. Coste previsto: una consulta.

AISLAMIENTO POR USUARIO: 11/11 EN EL ENTORNO LOCAL · EN PRODUCCIÓN SOLO SE HAN VERIFICADO LAS CUATRO FUGAS CERRADAS EL 8-OCT, NO LA BATERÍA ENTERA

EVIDENCIA DE EXPOSICIÓN Y DE LA FASE 1 COMPLETA

⚠️ **HUECO** — responsable: **el grupo**. Plan de pruebas formal y capturas de producto de los dos defectos ya corregidos (el error 500 de la comprobación de vida y la trampa del Spider): falta reproducirlos en el entorno local sobre el producto actual y capturar la pantalla.

Operación degradada (RNF-06), probada sin gastar saldo

La prueba que demuestra RNF-06 no necesita ni clave de API ni conexión: sustituye el cliente del modelo por uno simulado y comprueba qué devuelve el clasificador en cada situación. Ocho casos: los tres estados sobre la vía del proxy, el estado degradado en las cuatro vías, el camino correcto de la identificación de servidor, y que se le entrega un esquema de respuesta al cliente.

El valor de la prueba está en haberla visto fallar. Ejecutada contra el código anterior devuelve *0 de 8*, y el motivo es revelador: los cuatro métodos llamaban a .get() sobre el objeto que el cliente reescrito devuelve ahora, de modo que la fase de análisis no es que fallara en silencio, es que se interrumpía entera. Contra el código corregido devuelve *8 de 8*. Una prueba que pasa siempre no prueba nada. Esta distingue las dos versiones.

Reproducible con python3 ia/tests/test_tres_estados.py. Evidencia completa, con las dos salidas, en docs/evidencias/rnf06-tres-estados-antes-y-despues-09oct.md.

RNF-06 visto en el panel, que es donde tenía que verse

Una prueba unitaria demuestra que el clasificador **devuelve** el estado degradado. No demuestra que alguien lo **enseñe**. Entre las dos cosas hubo un hueco real: durante un día el estado existió en el módulo de IA y no lo consumía nadie al otro lado, de modo que RNF-06 estaba cumplido y era invisible. El 10-oct se cerró el hueco y se comprobó en la interfaz, con una auditoría lanzada desde el panel y sesión real.

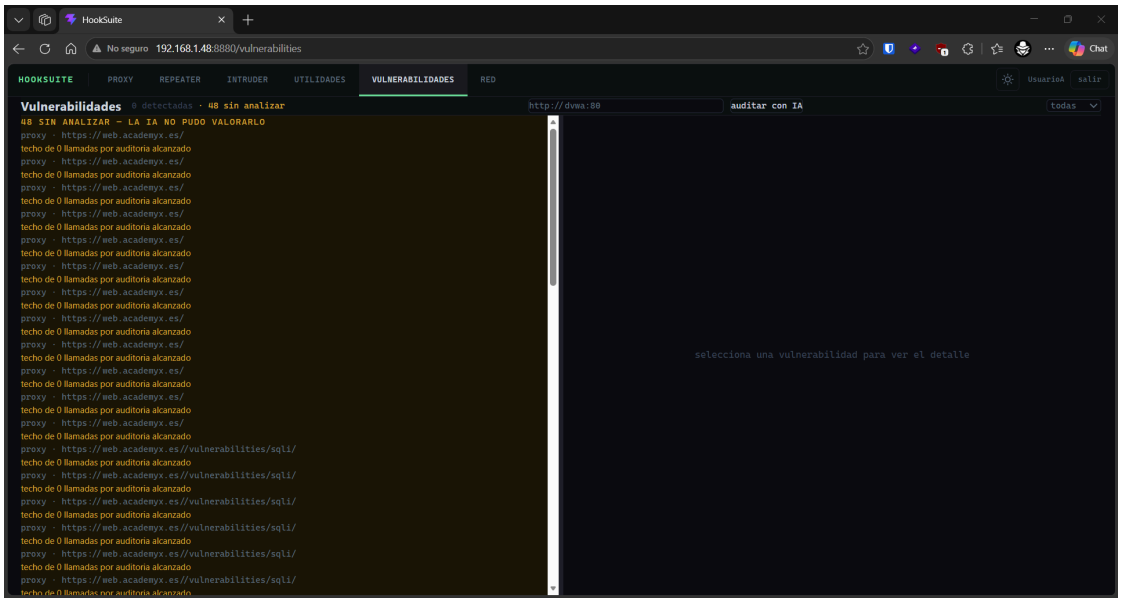


Figura 16: RNF-06 / RF-08 — el panel tras una auditoría truncada por el techo de gasto: **48 sin analizar, la IA no pudo valorarlo**, cada entrada con su motivo («techo de 0 llamadas por auditoria alcanzado») y el recuento «0 detectadas · 48 sin analizar» separando las dos cosas. Los elementos provienen de un rastreo previo sobre una web propia del equipo. El objetivo de la barra era el valor fijo que el panel traía entonces, sustituido el 10-oct por la herencia del objetivo rastreado.

Lo que la figura prueba, y que ninguna prueba unitaria alcanzaba: que el mensaje de lista vacía **dejó de mentir**. Antes, una auditoría que no había podido analizar nada y otra que había analizado y no encontrado nada producían la misma pantalla. Ahora el aviso va aparte de los hallazgos, en su propia tira, por encima, porque mezclarlo obligaría a inventarle una severidad a algo que precisamente **no se pudo valorar**.

Y el 15 de octubre volvió a verse, en producción y por un motivo distinto. Una auditoría lanzada sobre la instalación pública recorrió los cuarenta y ocho elementos de un rastreo previo y **ninguno** llegó a analizarse: el recuento quedó en «0 detectadas · 48 sin analizar · 40 analisis realizados», y en la lista de no analizados convivieron **dos** motivos, «clave de API invalida o ausente» en los intentos que salieron, y «techo de 40 llamadas por auditoria alcanzado». Tiene más valor que el caso de laboratorio precisamente porque **no se provocó**: la vía degradada se activó por una condición real del entorno, el cliente del modelo la devolvió **como estado y no como excepción**, y la pantalla la mostró separada de los hallazgos y con su motivo. El mismo mecanismo, dos causas distintas: eso es lo que RNF-06 pedía. De paso deja a la vista el valor del techo que gobierna la auditoría, cuarenta consultas, que el apartado 9 detalla.

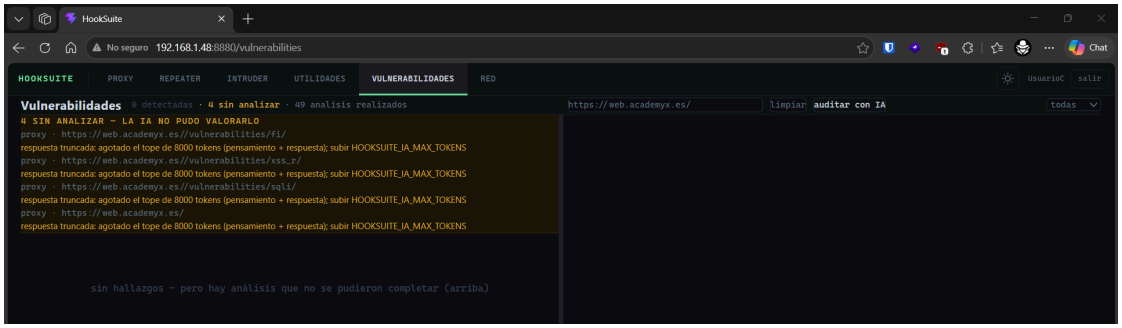


Figura 17: RF-08 / RNF-06 — **la tercera forma de quedarse sin análisis**, el 10-oct: cuarenta y nueve análisis realizados, cero hallazgos y cuatro elementos sin analizar porque la respuesta se truncó al agotar el tope de tokens. El mensaje en pantalla recomienda **subir** ese tope y el consejo es erróneo: el cliente no pide la respuesta por partes, de modo que subirlo hace fallar la consulta. La barra de pestañas es anterior al 13-oct.

Lo que el modelo puede devolver, y lo que el código daba por hecho (RF-08)

Las cuatro vías de análisis del módulo de IA piden al modelo una respuesta con un formato fijo, impuesto mediante un **esquema** que viaja con la consulta: la API garantiza que la respuesta se ajusta a él. El esquema no es documentación, **es el control**, y un campo declarado con tipo abierto no es un campo permisivo, es un campo sin control. La prueba comprueba que el esquema obliga a lo que las instrucciones piden, que la bandera de resultado sea un booleano y que la confianza venga en escala de 0 a 100. Contra el código anterior devuelve *5 de 11*, y los fallos no eran cosméticos:

1. De los tres campos de bandera, uno por vía, solo el primero estaba declarado como booleano. Los otros dos admitían cadena de texto, y el clasificador evalúa los tres dentro de un if: en el lenguaje en que está escrito, **cualquier cadena no vacía es verdadera, incluida la palabra que significa «no»**. Una respuesta negativa del modelo se habría registrado como vulnerabilidad confirmada.
2. La confianza se declaraba como número **sin rango**, así que nada rechazaba un valor fuera de la escala.
3. El esquema de la vía de identificación de servidor declaraba **cero campos**, mientras sus instrucciones piden nueve.

La causa era una sola y estaba fuera del repositorio del producto. Los esquemas los genera una herramienta de la práctica que deduce los campos a partir de las lecturas que el propio clasificador hace sobre la respuesta, y su lista de campos, escrita a mano, se redactó mirando **una sola** de las cuatro vías. El esquema vacío tiene la misma raíz por el camino opuesto: esa vía devuelve la respuesta sin inspeccionarla, y un esquema no se puede derivar de un código que no mira los datos. El arreglo fue en el generador, no en el fichero generado, y después se regeneró: la prueba pasa *11 de 11* y la de RNF-06 sigue en *8 de 8*. Un matiz queda como **limitación conocida**: la API no admite mínimo ni máximo en salida estructurada, de modo que el rango 0-100 tuvo que salir del esquema y un valor fuera de escala ya no lo para nadie. El caso peligroso, el de la escala 0-1, lo cubre la guarda del clasificador que se describe más abajo.

Reproducible con python3 ia/tests/test_esquemas_bandera.py ia/esquemas.py. Evidencia con las dos salidas, antes y después, en docs/evidencias/esquemas-bandera-antes-10oct.md.

Tres rechazos seguidos de la API: validar contra tu propia prueba no es validar (RF-08)

La primera auditoría **real**, clave válida, objetivo propio, techo de gasto puesto, no devolvió ni un hallazgo: las llamadas que salieron se rechazaron antes de procesarse, y hubo que lanzarla **tres veces** porque cada rechazo tenía un motivo distinto. Los objetos del esquema iban abiertos, y la salida estructurada no lo admite. La confianza declaraba un rango con mínimo y máximo, que tampoco se admite. Y veintidós campos declaraban su tipo como una lista de tipos posibles que incluía «objeto», lo que exige una declaración explícita que esos campos no llevaban. **Coste: cero**, porque un rechazo por petición mal formada no consume saldo. La factura fue de tiempo.

Lo que este episodio enseña, y es lo que justifica contarlo. La prueba de esquemas daba *11 de 11* **y las seguía dando con el defecto dentro**: comprobaba lo que nosotros le pedíamos al esquema y nunca lo que la API exige para aceptarlo, y ese requisito no estaba escrito en ninguna parte nuestra. Dicho corto: **un esquema validado contra tu propia prueba no está validado. Está de acuerdo contigo.** Quedan dos piezas para que no haya una tercera

vez. La prueba ia/tests/test_esquemas_api.py recorre los esquemas en profundidad y rechaza cualquier clave que no conste como admitida, verificada distinguiendo las dos versiones (*1 de 5* contra los esquemas defectuosos, *5 de 5* contra los corregidos) y con una comprobación de que sabe contar, para que un cero no se confunda con un aprobado. Y la herramienta ia/tests/validar_esquemas_contra_api.py somete cada esquema al servicio real y responde si pasa, los cuatro pasan, porque **cuando el contrato lo pone otro, se le pregunta a él**. Los dos defectos de esquemas nacieron en el mismo sitio, el generador, que es la única pieza del proyecto sin control de versiones: **lo que no tiene historial es donde nadie mira**.

Un fallo silencioso no es un fallo menor: la escala de la confianza (RF-08 / RNF-06)

El filtro por confianza compara el valor que devuelve el modelo contra un umbral. Las instrucciones piden ese valor en escala de 0 a 100, pero ninguna de las cuatro pide que sea un número entero, y ahí había una rendija que el esquema **no puede** cerrar: si el modelo devolviera la escala de 0 a 1, un 0,85 queriendo decir «85 %» está dentro del rango permitido, pasaría el control y quedaría por debajo del umbral. El hallazgo **desaparecería sin dejar rastro**, que es exactamente la situación que RNF-06 existe para evitar. El arreglo va en el clasificador y no en el esquema, porque imponer «entero» habría rechazado un decimal legítimo: la franja entre 0 y 1 se trata como **sospecha de escala** y se encamina al estado degradado de RNF-06, con su motivo, de modo que el problema se ve en el informe en lugar de desaparecer.

La prueba distingue las dos versiones, que es la única forma de que valga: contra el código corregido devuelve *19 de 19* y contra el anterior *6 de 19*, y los seis que pasan en la versión antigua son precisamente los casos legítimos, lo que demuestra que la guarda no introduce falsos positivos. Reproducible con python3 ia/tests/test_escala_confianza.py. Evidencia con las dos salidas en docs/evidencias/escala-confianza-guardia-10oct.md.

La interfaz antes de descartar RF-09, capturada antes de retirarla

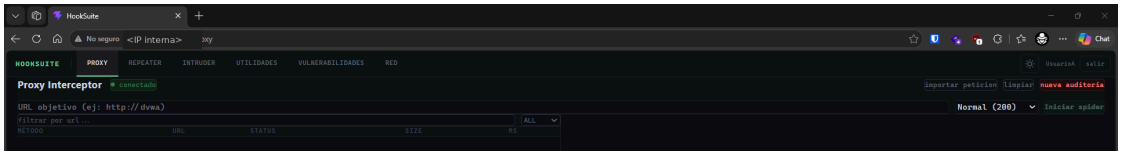


Figura 18: Barra de navegación de HookSuite con las **seis** pestañas, antes de retirar RF-09. «RED» es la última. Capturada en el entorno local el 11-oct, antes de aplicar el cambio. La dirección interna de la barra del navegador va censurada. Requisitos: RF-09 (descartado) y RF-01 (interfaz).

La figura existe porque **las evidencias de un defecto se capturan antes de arreglarlo**: retirar la pestaña hacía imposible volver a fotografiarla, así que se pidió la captura antes de tocar el código. De las tres evidencias previstas se tomó una sola imagen: las otras dos, el extremo de consola que responde 404, se guardaron como texto, que es buscable, comparable entre versiones y no arriesga publicar de refilón una credencial que estuviera en pantalla.

SECCIÓN 8

Matriz de trazabilidad

La pieza con la que se corrige la práctica: requisito por requisito, dónde está implementado, qué prueba lo verifica y dónde se ve. Las rutas están comprobadas en el código (4-oct). La columna de evidencia enlaza la figura de la memoria y el **minuto exacto del vídeo**. Se rellena al grabar.

Req.	Descripción	Estado	Prueba de implementación (ruta)	Evidencia
RF-01	Cumplido (seis pestañas: se retira Red y se añade Manual del 11-oct)	Cumplido	PR-01 frontend/src/App.jsx + Layout.jsx + pages/manual/ManualPage.jsx	fig. apdo. 9 . vídeo —: —
RF-02	Modificado (dos errores de servidor corregidos al inicio de la práctica)	Modificado	PR-19 backend/services/proxy_service.py	justif. apdo. 3 . apdo. 5
RF-03	Cumplido (cuatro defectos corregidos al inicio de la práctica. Quince más en el pulido del 11-oct. El 12-oct las cabeceras que muestra pasan a ser las realmente enviadas)	Cumplido	PR-03 backend/services/spider_service.py, frontend/…/ProxyPage.jsx, …/RequestsTable.jsx	figs. apdo. 7 . vídeo —: —
RF-04	**Modificado** (12-oct: el Repeater envía con cliente propio y la sesión se hereda por cabecera explícita)	Modificado	PR-04 backend/services/session_service.py, backend/services/	justif. apdo. 3 . figs. apdo. 7

	com- par- ti- da)	proxy_service.py, bac- kend/ rou- tes/ repeater.py	
RF-05	Cumplido (parser y avisos de error corregidos al inicio de la práctica. El importador de peticiones se traslada aquí desde el Spider el 11-oct y se **retira** el mismo día. Dieciséis defectos corregidos en el pulido del 12-oct, con un historial añadido y retirado esa misma jornada)	bac- kend/ rou- tes/ repeater.py, bac- kend/ ser- vi- ces/ proxy_service.py, fron- tend/ src/ pa- ges/ re- pea- ter/	figs. ap- do. 5 . ví- deo —: —
RF-06	Cumplido (cancelación corregida al inicio de la práctica)	bac- kend/ ser- vi- ces/ intruder_service.py	fig. ap- do. 5 . ví- deo —: —
RF-07	Cumplido	fron- tend/ src/ pa- ges/ uti- li- ties/	fig. ap- do. 5 . ví- deo —: —
RF-08	Parcial: el recorrido está verificado de punta a punta (10-oct) y el techo de gasto ejecutado, pero **ninguna auditoría ha devuelto todavía un hallazgo analizado**, todas las que han llegado al final salieron degradadas, por el techo de gasto, por la clave de API o por el tope de tokens	bac- kend/ rou- tes/ playwright.py, bac- kend/ ser- vi- ces/ redis_consumer.py, fron- tend/…/ VulnerabilitiesPage.jsx	figs. ap- do. 7 . ví- deo —: —
RF-09	**Descartado** (11-oct. Justificado en apartado 3)	dev- tools/ con- ser- va-	fig. ap- do. 7 .

	nel Red	do . bac- kend/ rou- tes/ network.py se man- tie- ne (lo con- su- men IA y Playw- right)	ví- deo —: —
RF-10	**Parcial** (construido y conectado a la red interna del despliegue. PR-10 sin ejecutar y el sondeo de instrucciones devuelve 401 con sesión válida, apartados 6 y 7)	PR-10 playwright/ + bac- kend/ rou- tes/ playwright.py	ap- dos. 6-7 . ví- deo —: —
RF-11	Cumplido (reimplementado al rehacer el acceso por usuario)	PR-11 bac- kend/ ser- vi- ces/ guardia.py + ser- vi- ces/ auth_service.py	ap- do. 6 . figs. ap- do. 7
RF-12	Cumplido (las cuatro fugas de la P1 cerradas y verificadas en producción. La batería de once casos de aislamiento, superada en el entorno local)	PR-12 bac- kend/ ser- vi- ces/ auth_service.py, guardia.py, usuarios_store.py, fron- tend/…/ LoginPage.jsx	ap- do. 6 . figs. ap- do. 7
RNF-01	Cumplido, con limitación documentada en móvil (apdo. 9)	PR-01 fron- tend/ ser- vi- do por Nginx	ap- do. 9

	talarnada en el cliente		
RNF-02	Cumplido (canal autenticado: el token viaja en el primer mensaje, no en la URL)	backend/main.py (/ws/)	apdo. 6. vídeo —: —
RNF-03	Cumplido, y desde el 15-oct con la **reproducibilidad probada**: instalación desde cero siguiendo el README, ejecutada hasta la pantalla de acceso	docker-compose.yml + infra/nginx.conf + README.md	apdo. 7. vídeo —: —
RNF-04	Parcial: el historial **no refleja un reparto de tareas, porque no lo hubo**, la autoría de cada confirmación se asigna por sorteo y se declara en el apdo. 10. Además de los correos del mismo autor parten el grafo de contribuciones	repositorio del producto	apdo. 10
RNF-05	Cumplido	backend/services/intruder_service.py (asyncio.Semaphore)	apdo. 7
RNF-06	Cumplido y **visto en el panel** (10-oct, y de nuevo el 15-oct en producción con un motivo distinto: clave de API caducada)	ia/tests/test_tres_estados.py (8/8), ia/tests/test_escala_confianza.py (19/19), ia/analyzers/vulnerability_classifier.py, backend/services/redis_consumer.py, fron-	fig. apdo. 7

	o no su- pera el um- bral	tend/…/ VulnerabilitiesPage.jsx	
RNF-07	Parcial: exposición cerrada, servidor endurecido, autenticación y aislamiento verificados, pero las dependencias acumulan avisos conocidos (12 en el frontend y 78 en el del backend, apdo. 6) y no se actualizó ninguna	PR-14, PR-16	apdo. 6-7
RNF-08	Cumplido en la P1 y endurecido después (permisos del canal y validación de entradas), pero **su ciclo por sesión, abrir al entrar y cerrar al salir o a las cuatro horas, no se ha vuelto a ejercitar en esta práctica**: PR-15 no tiene caso escrito	PR-15	apdo. 6-7
RNF-09	Pendiente	PR-18	apdo. 7

RUTAS VERIFICADAS EN CÓDIGO · PRUEBAS Y MINUTOS DE VÍDEO POR COMPLETAR

La columna de prueba remite al **plan de pruebas del apartado 7**, que reconstruye los dieciocho casos citados con dos resultados, el del 29 de septiembre sobre el prototipo y el de hoy sobre el código congelado, y declara caso por caso de dónde sale el resultado de hoy. La numeración RF/RNF es la fijada en el apartado 3.

⚠️ **HUECO** — responsable: **el grupo**. Al revisar el vídeo grabado, anotar el minuto exacto donde se demuestra cada requisito y el número de figura de su captura.

SECCIÓN 9

Limitaciones y trabajo futuro

Qué no funciona, qué funciona con condiciones y qué se haría con más tiempo. Este apartado recibe material hasta el último día.

El estado de sesión es volátil, por diseño

El estado de una sesión de auditoría, tráfico capturado, resultados del Intruder, vulnerabilidades, vive **en memoria del backend**: al reiniciarlo, se pierde. Es una **decisión consciente**, no un olvido, y se sostiene en tres razones:

1. **El patrón de acceso del backend no lo soporta sin reescribirlo.** Una treintena de puntos obtienen el diccionario de la sesión y lo mutan en memoria (session[«requests»].append(…)). Persistir de verdad obligaría a escritura-a-través en cada mutación, es decir, a tocar todo el backend a pocos días de la entrega.
2. **El modelo de sesión cambió al rehacer el acceso.** El login por usuario ata las sesiones a identidades. Lo que se persistiera con el modelo anterior habría que rehacerlo.
3. **Una sesión de auditoría es un espacio de trabajo efímero.** El operador lanza un rastreo, examina los resultados y exporta lo que importa. No espera que la herramienta recuerde una sesión de hace tres días.

Lo que **sí** se hizo, porque era el riesgo real detrás de «todo en memoria»: **poner topes**. El recolector de sesiones existía en el código pero nunca se invocaba, y cualquier llamada a la API con un identificador nuevo creaba una sesión más, con la API sin autenticación, eso permitía agotar la memoria del servidor desde fuera. Ahora hay un tope de sesiones y de peticiones por sesión (configurables por entorno), el recolector se ejecuta de verdad y, al desalojar, **prefiere las sesiones que no tienen conexión en tiempo real abierta**, para no interrumpir a quien está trabajando. Verificado con prueba de abuso: 120 identificadores inventados dejan el número de sesiones en el tope, y la sesión con conexión viva sobrevive.

Trabajo futuro. La persistencia es viable sin cambiar la arquitectura: el despliegue ya incluye Redis y el backend ya lo usa para otro flujo. El momento correcto es **después** del modelo de usuarios, para persistir sesiones ya asociadas a su propietario.

El techo de gasto de la IA: aplicado, y lo que sigue sin estar acotado

Lo que hay hoy. El cliente del modelo lleva un **techo de consultas por auditoría**, situado ahí y no en el orquestador porque por el cliente pasan las cuatro vías de análisis. Su valor por defecto es de **cuarenta consultas**, configurable por entorno (HOOKSUITE_IA_MAX_LLAMADAS), y al agotarse devuelve la vía degradada de RNF-06: la auditoría queda marcada como **truncada** en vez de parecer completa, porque un techo silencioso dejaría el informe diciendo «sin hallazgos» sobre una auditoría a medias. El 15 de octubre se comprobó en la máquina de producción que el recorte llega hasta la pantalla con sus propias palabras, «techo de 40 llamadas por auditoria alcanzado», elemento por elemento.

Lo que sigue siendo limitación, y por eso este apartado no desaparece. El techo cuenta **consultas y no dinero**, y lo hace **por auditoría**: acota el caso peor de una, no el gasto de

una jornada. Y el valor de cuarenta entró como **provisional**, el criterio previsto era fijarlo con el consumo de la primera auditoría real, y **sigue siéndolo**: no se ha reajustado.

Lo que queda de la confusión de escala, una vez cerrada

Lo que **sí** queda, y es consciente:

1. **Un valor legítimo de 1 se marca como sospechoso.** La guarda no puede distinguir «1 %» de «certeza total en escala de 0 a 1», porque son el mismo número. Se eligió marcarlo: con el umbral en 60 ese valor tampoco se habría reportado, de modo que el coste es un aviso de más y la ganancia es que el caso peor, descartar un hallazgo seguro, deja de ser posible. **Preferir el falso positivo visible al falso negativo silencioso** es la misma elección que ya tomó RNF-06.
2. **La vía de identificación de servidor no lleva guarda**, porque no compara contra el umbral: entrega la respuesta del modelo sin filtrar, así que no hay descarte silencioso que cerrar.
3. **No se cambiaron las instrucciones del modelo para pedir un entero**, que era la alternativa más limpia en el plano conceptual, instrucciones y esquema de acuerdo en un contrato más estricto. Se descartó por el momento: son cuatro ficheros del material que no se vuelca a la conversación, se editarían sin verlos, y son prosa y no una línea de código, a tres días de la congelación. Queda como trabajo futuro de coste bajo.

El manual de uso va por dos pestañas escritas y tres por escribir

El 11-oct se añadió una sexta pestaña, **Manual**, con el modo de uso de cada herramienta. Ataca de raíz el patrón descrito en el apartado 7, funciones correctas que no se ven, y por eso no es documentación accesoria sino parte del producto.

Están escritas **dos secciones de cinco**: la del Spider, con los veinticinco controles de esa pantalla detallados uno a uno, de qué tipo es cada elemento (botón, campo de texto, desplegable, aviso, indicador, pestaña, columna), qué hace y **bajo qué condición aparece**, porque buena parte no está en pantalla desde el principio, y la del Repeater, con veintiséis, añadida el 12-oct en cuanto esa pestaña quedó pulida. Desde entonces el manual se lee **por pastillas** y no en una tirada, y las tres secciones sin escribir se marcan a la vista del usuario, que era la decisión del 11-oct: si falta, se ve que falta.

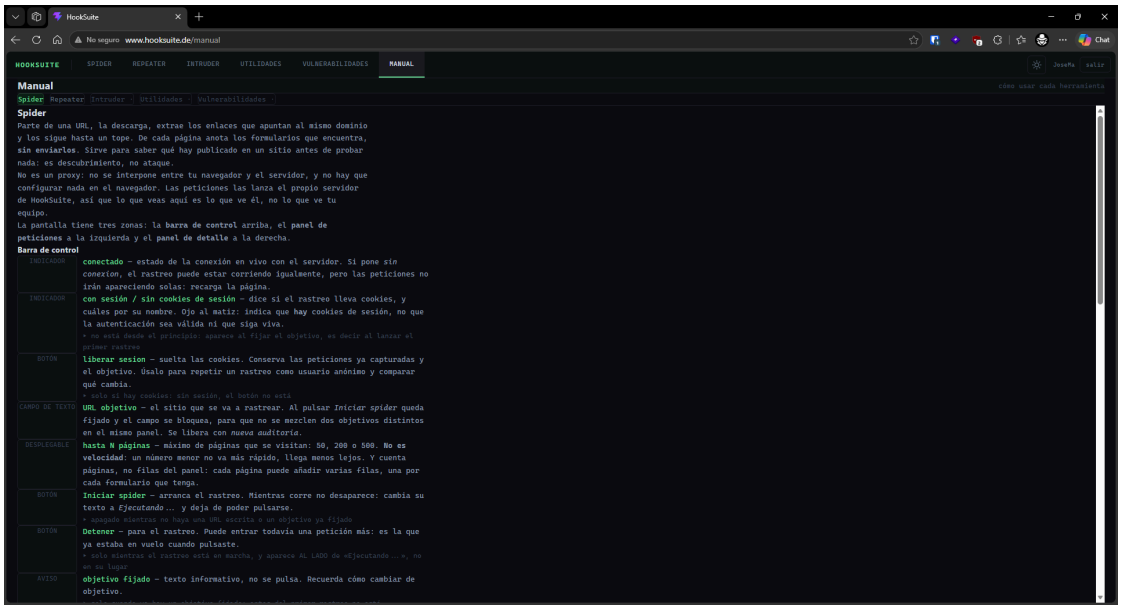


Figura 19: RF-01 — la pestaña Manual: cinco pastillas, la activa resaltada y las tres secciones sin escribir marcadas con un punto, de modo que el usuario ve que el manual está incompleto. Debajo, el modo de documentar: cada control con su tipo (indicador, botón, campo de texto, desplegable, aviso), qué hace y bajo qué condición aparece.

La regla que se fijó es deliberada: **una sección se escribe cuando su pestaña se ha pulido y verificado**, nunca antes. Durante el pulido del Spider, lo que suponíamos de la pantalla y lo que el código hacía no coincidieron quince veces. Un manual redactado sobre suposiciones induce a error, que es peor que no tenerlo. La propia redacción lo ha confirmado tres veces: destapó el defecto de los filtros (apdo. 7). Un error del manual mismo, que afirmaba que el botón «Iniciar spider» se sustituía por «Detener» cuando en realidad cambia a «Ejecutando…» y «Detener» aparece al lado. Y, al escribir la sección del Repeater, veinte líneas de **código muerto**, un aviso que ya no podía aparecer nunca, porque lo provocaba el historial retirado esa misma tarde. Documentar obliga a mirar el código con los ojos de quien lo usa, y eso encuentra cosas que el uso no encuentra.

Queda pendiente decidir si el manual lleva capturas. Se montó el soporte y se descartó: las capturas de evidencia disponibles son recortes pensados para demostrar un arreglo, no para enseñar a usar, y las nuevas hay que tomarlas con ese propósito.

Dos cosas del Spider que se dejan dichas, no hechas

- **Cuántas veces aparece un formulario.** Un formulario se cataloga una sola vez, bajo la primera página donde se encontró. Es lo correcto para no inflar el recuento, pero se pierde un dato útil para el auditor: en cuántas páginas está presente. La etiqueta actual dice «vistos aquí primero», que avisa del comportamiento sin dar el número.
- **La etiqueta del desplegable.** Se corrigió para que no insinuara más de lo que hay, y quedó anotada para revisarla en la fase de diseño.

El Repeater trabaja con una petición cada vez, y es una decisión

El Repeater no tiene historial: mantiene **una** petición cargada y, mientras la tenga, el Spider no permite enviarle otra, su botón pasa a «repeater en uso» y explica cómo liberarlo. Se cuenta aquí porque la ausencia de historial es lo primero que echará en falta quien conozca la herramienta de referencia, y porque el camino hasta aquí explica la decisión mejor que la decisión misma: **el historial existió durante nueve horas del 12-oct y se retiró** (apartado 7).

Por qué se sostiene como diseño y no como carencia. El desorden que motivó la retirada venía de tener varias peticiones conviviendo sin que nada dijera cuál era la buena: un estado **implícito**. Ahora hay una sola, el estado ocupado se ve **desde la otra pantalla** y se libera con un gesto deliberado. Para una herramienta cuya salida acaba en un informe, que no se pueda confundir qué respuesta pertenece a qué petición vale más que poder recuperar la de hace diez minutos.

Trabajo futuro, con el camino ya identificado: una lista propia en la sesión del servidor, separada de la que comparten el Proxy y el Spider. La vía que se descartó para esta entrega era añadir un campo de origen al registro **compartido** por esas dos herramientas y filtrar en el extremo del servidor: toca código común a tres pantallas y obliga a recrear el servidor, que borra las sesiones en memoria. Es trabajo para después de la entrega.

El borrador del Repeater guarda las credenciales del objetivo en el navegador

Limitación conocida y declarada, detectada el 13-oct al destilar la jornada anterior. El editor del Repeater guarda lo que estás escribiendo, método, dirección, cabeceras y cuerpo, en el almacenamiento de sesión del navegador, para que no se pierda al cambiar de pestaña o refrescar. Ese guardado **no filtra las cabeceras de sesión**, así que la Cookie del sitio auditado, credenciales de un tercero, queda escrita en el navegador del auditor.

Cómo llegó a estar así, porque el recorrido es la parte aprovechable. El 12-oct a las 12:37 se añadió exactamente esa protección, y se añadió donde hacía falta entonces: en la persistencia del **historial**. A las 17:28, al retirar el historial, la función de filtrado se fue con él. Lo que sigue escribiendo en el navegador es el **borrador**, que nunca pasó por ese filtro. Dicho de otro modo: **la protección no se eliminó por error, desapareció con la función a la que estaba pegada**. Conviene retener el patrón, una medida de seguridad acoplada a la función que la motivó se pierde cuando esa función se retira, porque no lo detecta ninguna prueba: el producto sigue funcionando igual.

Riesgo real, acotado sin minimizarlo. El almacenamiento de sesión es legible por cualquier código que se ejecute en la página, de modo que aprovecharlo exige antes una ejecución de código en el propio panel. La única superficie de ese tipo que el producto pinta, la vista previa de la respuesta del objetivo, está aislada desde el 11-oct, y el almacenamiento se vacía al cerrar la pestaña. No es una puerta abierta: es un dato que no debería estar ahí.

Por qué no se arregla ahora, y qué costaría. La corrección son tres líneas: reutilizar al guardar la misma lista de cabeceras de sesión que el editor ya usa para avisar de que la petición viaja autenticada. Se decide no tocar el producto el día de la congelación. La contrapartida del arreglo también está medida: tras un refresco la petición volvería sin sesión y habría que pulsar «preparar» otra vez, que es la misma contrapartida que el grupo ya aceptó el 12-oct para el historial.

Y dos detalles del mismo borrador, menores pero medidos, para que no se lean como un olvido. El borrador se **relee del almacenamiento en cada repintado** de la pantalla, aunque solo haga falta la primera vez, se vuelve a interpretar el mismo texto para descartarlo acto seguido. Y «limpiar» borra la clave del almacenamiento, pero el guardado automático se dispara inmediatamente después, al vaciarse los campos, y **la vuelve a escribir con la petición en blanco**: no queda nada del trabajo anterior, pero la clave reaparece. Ninguno de los dos afecta al uso. Se dejan escritos porque se encontraron leyendo el código y van al mismo sitio que su arreglo cuando se retome.

Lo que falta para igualar al Repeater de referencia

Tres diferencias conocidas y no cerradas:

- **No hay botón para seguir una redirección.** El Repeater muestra el 3xx y no lo sigue, que es el comportamiento correcto. La herramienta de referencia ofrece además seguirla con un clic. Es trabajo de interfaz y de un extremo nuevo en el servidor, y quedó fuera por la fecha.
- **El origen de las peticiones, que es la diferencia de fondo y no está en el Repeater.** En la herramienta de referencia el formulario llega **capturado del navegador**, con los datos que el auditor escribió y un token recién emitido: una petición que ya funcionaba. Aquí llega una **plantilla** leída del HTML por el Spider, que nunca se envió. El botón «preparar» cierra la distancia a mano, pide la página y refresca los campos ocultos y la cookie, pero no la elimina: eliminarla exige capturar el tráfico real del navegador, que es justamente lo que se descartó con RF-09.
- **Las cabeceras repetidas se muestran unidas por coma**, no como dos filas, porque el panel de detalle espera un objeto. Se ven, antes se perdía una, pero no se editan por separado.

El frontend no pasa su propio analizador estático

El repositorio tiene configurado un analizador estático para el frontend y, hasta el 12-oct, **nunca se había ejecutado** (apartado 7). Puesto a funcionar, arroja **veintitrés errores y dos avisos** que no son del trabajo de estos días: casi todos de reglas del marco de interfaz, efectos que escriben estado, funciones con efectos secundarios donde se esperan puras, y concentrados en la pantalla de vulnerabilidades. Lo que sí quedó cerrado es la regla que caza el fallo que costó la regresión del botón de enviar: no queda ninguna variable no declarada en todo el frontend.

Por qué no se arreglan ahora: son avisos de calidad sobre pantallas que funcionan y ya están capturadas, y tocarlas el día de la congelación arriesga el entregable sin mejorar el producto. Queda como **trabajo futuro con el dato ya medido**, que es lo que permite planificarlo en vez de prometerlo. Y una limitación del método, dicha entera: el analizador solo puede ejecutarse dentro de la etapa de construcción de la imagen, porque el proyecto no instala las dependencias del frontend en el servidor.

Restos del laboratorio retirado

El laboratorio vulnerable salió del producto el 13-oct (apartado 6), pero **su nombre sigue en dos sitios que no se ejecutan**: la función de acceso del módulo de navegador (RF-10, parcial) y los datos de ejemplo del frontend, que **sí viajan en el paquete** que se sirve al navegador aunque solo se usen cuando no hay servidor detrás. No es un riesgo, los valores son inventados, pero el entregable menciona un componente que ya no incluye. Es limpieza de diez minutos que no se hizo el día de la congelación a propósito: tocar código que funciona, el último día, para arreglar un nombre, es justo el cambio que no compensa.

La herramienta que genera los esquemas no está versionada

El generador de esquemas vive en la carpeta de trabajo de la práctica, **fuera del repositorio del producto**: no tiene historial, no pasó por revisión y su autoría no queda registrada, al contrario que todo el código. Es incoherente con la norma del equipo de que todo entra por petición de incorporación, y no es una incoherencia inocua: **el único sitio sin historial resultó**

ser exactamente donde estaba el defecto del apartado 7, porque es el sitio donde nadie vio la lista incompleta.

Trabajo futuro, de coste bajo: mover las herramientas de la práctica al repositorio, para que un cambio en ellas se revise como cualquier otro. No se hizo ahora porque mover ficheros a tres días de la congelación introduce riesgo sin aportar nada al producto.

⚠️ **HUECO** — responsable: **el grupo**. Acumular las limitaciones que aparezcan hasta el cierre de la entrega. Este apartado recibe material hasta el último día.

SECCIÓN 10

Reparto del trabajo

Quién hizo qué, con estimación de horas por persona.

Cómo se repartió el trabajo, y por qué el historial no lo refleja

El producto es de los cinco. HookSuite lo diseñó y lo construyó el grupo entero en la Práctica 1, interfaz, servidor, navegador real, captura de tráfico y módulo de IA, un módulo por persona, y **ese** es el producto que se entrega aquí. La Práctica 3 no construye una herramienta nueva: coge la de la P1 y la lleva a un estado ejecutable, seguro y reproducible. Todo lo que esta memoria cuenta descansa sobre el trabajo de los cinco.

El plan de trabajo también es del grupo. El alcance de esta práctica se decidió en la **reunión inicial del 29 de septiembre**: el grupo eligió abordar los cinco primeros puntos del plan de mejoras que la propia P1 había dejado escrito. Ese plan se desarrolló en un documento que se presentó al grupo el 2 de octubre, y recibió el visto bueno con tres reparos a esa primera versión que se atendieron uno a uno y quedaron incorporados. Las fases, el orden del trabajo y lo que se descarta salen de ahí.

Lo que no se pudo hacer en grupo fue la ejecución. Entre el trabajo de cada uno, las clases del máster y el día a día, al grupo le resultó **imposible** trabajar de forma presencial o continuada durante estas semanas. La práctica se ejecutó en sesiones de trabajo **con la ayuda de un asistente de IA**, el apartado 11 lo declara con detalle: todo el código añadido o modificado en esta práctica lo escribió el asistente, con el plan del grupo como guía y consultas al grupo en las decisiones de alcance.

Y las confirmaciones del repositorio no indican quién escribió cada línea. Se reparten al azar, a propósito, y lo que viene a continuación explica el mecanismo y su única excepción. Se declara aquí porque el enunciado dice expresamente que el historial de confirmaciones es una de las fuentes con las que se juzga el reparto: en este caso, usarlo para eso daría una foto equivocada.

Qué se pierde y qué se gana diciéndolo. Se pierde la comodidad de un historial que parece un reparto limpio. Se gana que la pregunta quede bien planteada: una confirmación con un nombre **no** es una afirmación de que esa persona escribiera ese código, y no debe leerse así. Lo que el grupo conoce, y lo que el enunciado pide que todos conozcan, es el producto y las decisiones que lo llevaron a este estado.

Y lo que esta memoria no va a decir, aunque sonaría mejor: que el grupo pudo trabajar de forma fluida y repartida en esta práctica. No ocurrió. Escribirlo convertiría una limitación de calendario, comprensible, ajena al producto y que no cambia lo que la herramienta hace, en una afirmación falsa sobre el equipo.

El mecanismo del sorteo de confirmaciones

La autoría de cada confirmacion se asigna **al azar** entre las cinco personas del grupo, y se declara aquí porque **el historial no dice quién hizo qué**. El motivo es que en esta práctica no hubo un reparto de tareas que reflejar: las decisiones se fueron tomando en **consulta de grupo continua**, dudas sobre la herramienta y cómo abordarlas, resueltas sobre la marcha, y la ejecución pasó por una sola vía, de modo que **ninguna confirmación es atribuible a**

una persona. Firmarlas todas a un mismo nombre habría dado una foto igual de equivocada sobre un producto que es de los cinco. Se eligió el sorteo, y se dice que es un sorteo. Con una excepcion fija: **la memoria tecnica se firma siempre a nombre de la misma persona**, porque es un documento unico que se regenera entero en cada cambio.

Esa excepcion deforma el recuento, y el dato concreto lo demuestra: de las 41 confirmaciones de la jornada del 10 de octubre, una persona figura en 19, mas del doble que cualquier otra, pero **11 de esas 19 son regeneraciones de la memoria**. Descontada la excepcion quedan 8, que es justo lo que cabe esperar de un sorteo entre cinco sobre 41. El reparto es uniforme. Lo que no es uniforme es el documento.

La estimacion de horas de este apartado sale del diario, no del historial de confirmaciones.

Dato del 12-oct, tomado del diario y no estimado a posteriori: la jornada suma **unas diez horas y media de asistente** repartidas en veinticinco entradas, más el tiempo de pruebas humanas, que **no se cronometra** porque va entremezclado con el uso de la herramienta. Al consolidar conviene no fundir las dos cifras: las horas del asistente están anotadas entrada por entrada. Las de prueba humana, no. Y el reparto de mérito no sigue al reloj, de los dieciséis defectos de esa jornada, la mayoría los encontró quien usaba el producto, no quien escribía el código.

Horas registradas: qué parte es medida y qué parte inferida

El diario tiene **179 entradas**, **177** con campo de horas, y suma **87,2 h**. El reparto de abajo separa a propósito las tres cosas que suelen ir mezcladas en una tabla de horas: lo que está **escrito**, lo que está **inferido con una regla declarada**, y lo que simplemente **no se registró**.

Quién	Horas	De dónde sale esa cifra
Claude Code (asistente)	**87,1 h**	**40,7 h escritas** con su nombre en el campo de horas, todas del tramo final de la práctica · **46,4 h inferidas**: entradas anteriores cuyo campo dice solo « 1 h», sin nombrar a nadie
José María	**≈1 h de la reunión, más guía continua no cronometrada**	La reunión del 29 de septiembre, como el resto del grupo. A partir de ahí, la guía de la ejecución sesión a sesión, que **no se cronometró** porque va entremezclada con el uso de la herramienta, y que se apoyó en esa consulta de grupo continua: las decisiones que trasladaba salían de ahí. Lo que sí se puede contar son sus intervenciones: **43 entradas** las registran, 23 decisiones, 5 revocaciones de claves, 5 pruebas en producción, 5 lotes de capturas, 3 ejecuciones en la instalación pública y 2 fusiones con permiso de administración
Ivan · Macarena · Nacho · Carlos	**≈1 h cada uno, más consulta continua no cuantificable**	La **reunión inicial del 29 de septiembre**, donde se acordó el plan de trabajo, duró aproximadamente una hora y la hicieron los cinco. A partir de ahí hubo **consulta de grupo continua** durante todo el proceso, en mensajes sueltos sobre la herramienta y cómo abordar cada duda: es real, pero **no se puede cronometrar ni repartir por persona**, y aquí no se inventa un número. Lo que no hubo fue reparto de tareas

La regla de inferencia, dicha para que se pueda discutir. En ese tramo final el campo escribe « **0,5 h de Claude**». Antes escribía « **1 h**» a secas. Las entradas las redactó **el mismo autor de principio a fin**, con el mismo formato y el mismo propósito, medir el tiempo de la sesión, así que **se infiere que el campo significaba lo mismo antes de decirlo**. No es una medida y no se presenta como tal: son **46,4 de las 87,2 h**, más de la mitad, y por eso la cifra del asistente se da como **87,1 h** con su desglose a la vista en lugar de como un total redondo. (40,7 + 46,4 = 87,1. La décima que falta hasta 87,2 es redondeo de cada entrada.)

Lo que esta tabla NO es, y conviene decirlo antes de que alguien la lea mal. No es el esfuerzo del grupo en el producto: HookSuite lo escribieron los cinco en la práctica anterior, y este diario solo cubre las sesiones de la P3, las de trabajo con el asistente, que es donde se hizo el trabajo de esta entrega. Tampoco mide valor: una decisión de treinta segundos, «DVWA fuera del producto», «las credenciales no van al repositorio», cambió el entregable más que varias horas de construcción.

Y el límite honesto: fuera de esa hora de reunión, el tiempo del grupo **no está en ningún registro**: ni el de la consulta continua ni el de la guía. No se cronometró, y aquí no se inventa.

SECCIÓN 11

Uso de herramientas de IA

Declaración obligatoria de qué herramientas de IA se usaron y para qué.

- **Claude Code** (modelo Claude de Anthropic): asistencia en diagnóstico, montaje del entorno local de pruebas, ensayo de limpieza del historial y redacción de esta memoria, bajo revisión humana.

Los límites de la herramienta, observados en primera persona

Usar un asistente de IA para auditar seguridad tiene un límite que conviene declarar porque lo encontramos trabajando: **el filtro de seguridad del propio modelo bloquea a veces tareas legítimas de este dominio**. El 9 de octubre, leyendo las instrucciones del clasificador de vulnerabilidades de HookSuite, la petición fue rechazada con un error que lo dice con todas las letras, el filtro «a veces marca tareas legítimas de programación, **ciberseguridad** y biología». No es un fallo del proyecto ni del operador: es el coste de trabajar en seguridad ofensiva con una herramienta generalista, y conviene contarlo con la fecha y la referencia delante en lugar de omitirlo.

Lo caro no fue el bloqueo, sino **la recuperación**. Para desatascar la sesión se usó la función de rebobinado del cliente, eligiendo entre sus cuatro opciones la que descarta lo posterior al punto de retorno. Se perdieron así ocho minutos de trabajo. Las otras dos opciones útiles, resumir desde el punto, o resumir solo lo anterior y permanecer al final de la conversación, habrían conservado ese trabajo. Es una lección de herramienta, no de ingeniería, pero cuesta horas igual.

De ahí sale el hallazgo que sí afecta al diseño de nuestros controles: **el rebobinado restaura la conversación y los ficheros, pero no los efectos sobre el servidor**. En la lista de puntos de retorno, el turno que había construido una imagen de contenedor figuraba como «sin cambios de código», porque esa función contabiliza ficheros. La imagen siguió existiendo después de rebobinar. Quien la construyó, el asistente, ya no recordaba haberlo hecho, y el único testigo del cambio fue un control externo que compara el estado del servidor contra lo que la documentación afirma. La consecuencia práctica es que **un control que mira ficheros no sustituye a uno que mira el sistema**, y que los avisos molestos de ese control no se pueden desactivar sin perder exactamente aquello para lo que sirve.

Qué fue generado y qué fue verificado. La distinción no es teórica: ese mismo día, una afirmación escrita por el asistente en el plan del módulo de IA, que los cuatro conjuntos de instrucciones pedían un campo descripcion, resultó falsa para uno de ellos al releer el código fuente, y se corrigió. El criterio que seguimos es que **nada entra en esta memoria por haberlo dicho la herramienta**: entra cuando se ha contrastado contra el código, la salida de un comando o el enunciado.

Los defectos que introdujo el asistente al arreglar otros

De los quince defectos del pulido del Spider (11-oct, apdo. 7), **dos los causamos nosotros**, y los dos comparten forma: un arreglo correcto en lo suyo que rompe algo contiguo que nadie volvió a mirar.

- **Los formularios pasaron de sobrar a faltar.** Al corregir que un formulario colgara de páginas ajenas, se ató cada formulario a su página de origen. Pero la página se registra con la URL **final**, tras redirecciones, y el formulario se ató a la **pedida**: cuando no coincidían, el formulario no colgaba de ninguna parte y desaparecía del panel. El defecto anterior era visible (sobraban filas). El nuevo era invisible (faltaban). Lo encontró una persona buscando un formulario concreto entre cincuenta y cinco páginas. El asistente **había avisado del riesgo y no lo comprobó**: avisar no sustituye a verificar.
- **Las últimas filas dejaron de alcanzarse.** Al bajar las acciones a una barra propia, el panel pasó de uno a dos hijos y la tabla siguió pidiendo el 100% de la altura: desbordaba, y el recorte se comía las primeras filas capturadas.

El 12-oct la cuenta subió a ocho en una sola jornada, y lo que importa no es el número sino qué los detectó:

Nº	Qué rompió el asistente	Qué lo detectó
1	Los escapes de un aviso nuevo se mostraban en crudo en la pantalla	los ojos de una persona leyéndolo
2	La dirección de destino se colocó en una barra sin sitio: solo se leía cuando la petición fallaba	una captura de producto
3	Un comentario colocado entre los atributos de un componente rompió la construcción	la construcción de la imagen
4	El arreglo de las cabeceras del Spider se aplicó a uno de los dos sitios, y todos los POST empezaron a dar 419	usando el producto
5	Un borrado por rango de líneas se llevó la declaración de una variable y el botón de enviar dejó de hacer nada, **sin error visible**	usando el producto
6	Al permitir ver el formulario sin haber enviado nada, siete usos que daban la respuesta por segura tumbaron la pantalla entera	usando el producto
7	La pestaña del formulario se pintaba sin estilos, y «limpiar» no la borraba	usando el producto
8	Un borrado de bloque cortado por el primer cierre dejó nueve líneas huérfanas y tres variables sin uso	una comprobación posterior del propio asistente

El reparto es el hallazgo. Seis de los ocho los encontró una persona **usando** el producto. Uno lo cazó la construcción de la imagen. Uno, una comprobación que el propio asistente hizo **después** de dar el cambio por terminado. Y **ninguno** lo cazó el método con el que el asistente verificó sus cambios durante toda la jornada, contar cadenas de texto dentro del paquete construido. Ese método confirma que algo **está**: no dice que se lea bien ni que funcione.

Dos de los ocho son el mismo error repetido el mismo día: borrar código por rango de líneas sin comprobar qué quedaba al lado. La primera vez se llevó por delante la declaración de una variable. La segunda, el cierre de un condicional anidado. De ahí sale una regla concreta, que vale más que la buena intención de «revisar mejor»: **los bloques se delimitan por contenido, no por el primer cierre que aparezca, y después se comprueba qué ha quedado huérfano**.

Lo que se saca de aquí es que la velocidad del asistente para aplicar un cambio no va acompañada de la misma velocidad para comprobar sus efectos laterales, y que **sus verificaciones son sistemáticamente ciegas a lo que ve una persona en dos segundos**. Por eso las capturas de producto las toma una persona y por eso el reparto del equipo pone las pruebas en manos humanas: no es desconfianza, es que los dos métodos fallan en sitios distintos.

Leer sin leer: delegar la lectura de lo que no debe entrar en la conversación

El filtro descrito arriba obligó a una medida de método que vale la pena declarar, porque cambió la forma de trabajar. Las instrucciones que el producto envía al modelo son el material que más incomoda a ese filtro y, a la vez, eran lo único que faltaba por conocer para terminar el

módulo de IA: **tres defectos abiertos dependían de saber qué formato piden**. El asistente principal no podía abrirlas sin arriesgar la sesión, y el permiso del operador no sirve de nada, porque el filtro no sabe nada de ese permiso: mide lo que entra en la conversación, no lo que alguien ha autorizado.

La salida fue **delegar la lectura**. Un asistente secundario, con instrucciones propias y un contexto que se destruye con él, abre los ficheros y devuelve solo su **interfaz**: nombres de campo, tipos, escalas y números de línea. No devuelve ni una frase del texto. El encargo se redactó con una precisión que resultó ser la clave, **«no los cites», nunca «no los leas»**, porque una versión anterior, mal formulada, llevó al secundario a **medir** los ficheros sin abrirlos y a contestar a medias.

De ahí salieron los tres defectos del apartado 7. Es decir: el control que impide volcar material delicado en la conversación no solo no frenó el trabajo, sino que obligó a una forma de trabajar, preguntar por la interfaz y no por el contenido, y comprobar con pruebas en vez de con la vista, que resultó **más** estricta que la lectura directa. Lo que no se puede leer, se mide, se edita a ciegas y se verifica con una prueba.

El límite del método también se declara: el asistente secundario es un modelo, no un control. Lo que devuelve tiene que revisarlo **una persona que conozca el material**, porque es la única que puede juzgar si cruzó la línea entre describir y reproducir. El asistente principal no puede, precisamente porque no conoce el contenido.

Qué aportó el asistente, fase por fase

El modelo de trabajo lo fijó el grupo el 1 de octubre: **se le ceden al asistente los mandos de construcción**, y lo que no puede hacer se reparte a personas y queda escrito, probar el producto en marcha, revocar claves, autorizar lo irreversible. El recuento de la última columna sale del diario, contando sus entradas una a una.

Fase	Qué construyó el asistente	Qué fue decisión o verificación humana	Registro
0 — Reconciliación e historial	Ensayo y ejecución de la reescritura del historial, el orden del empuje forzado de las siete ramas y la reconciliación de los ficheros sin confirmar	**La revocación de las claves**, repartidas en tres cuentas distintas: no es algo que el asistente pueda hacer. El visto bueno al empuje forzado y la decisión sobre la interfaz que solo existía en la instalación pública	9 entradas · 4,6 h
1 — Exposición y endurecimiento	Retirada de los puertos directos, orígenes permitidos, canal en tiempo real autenticado, topes del estado de sesión y endurecimiento del servidor	**Probar la web en producción**: nueve pruebas, tres hallazgos y una corrección al asistente. Y lanzar lo que los controles del servidor le frenan	19 entradas · 9,0 h
2 — Identidad y aislamiento	Alta y acceso por usuario, el guardián de los espacios de datos, las cuatro fugas y la quinta que introdujo la propia corrección	**El modelo de alta**: que no fuera abierto, sino con código de invitación. Y la verificación en la instalación pública	21 entradas · 15,9 h
3 — IA, pulido y congelación	RF-08 de punta a punta (bus interno, disparo desde el panel, techo de gasto, operación degradada), quince defectos del rastreador y dieciséis del banco de peticiones, el manual de	**Encontrar la mayoría de los defectos usando el producto**, seis de los ocho de una sola jornada, arriba, las capturas de pantalla, la clave de la API, la fusión con permiso de administración y las decisiones de **qué**	118 entradas · 47,1 h

	uso, la congelación y esta memoria	**no se toca** el día de la congelación	

Generado frente a verificado: la regla, y lo que cuesta sostenerla

- **Todo el código añadido o modificado en esta práctica lo escribió el asistente**, el de la práctica anterior lo escribió el grupo, y **nada se dio por bueno por haberlo escrito**. Lo que lo valida es otra cosa: sondas con valor esperado y **control negativo**, pruebas en el entorno de trabajo, y la verificación en la instalación pública, que la hace una persona.
- **Las cifras de esta memoria salen de medidas con herramienta, y donde no se ha medido, se dice.** El apartado 6 es el ejemplo: se da el número de avisos que da el auditor de dependencias y se declara, en la misma línea, que **no se ha medido** cuántos son alcanzables desde el camino de ejecución.
- **Las dos formas en que este asistente se equivoca, nombradas porque se repiten.** La primera es **un medidor que no sabría cantar otra cosa**: un cero sobre un PDF cuyo texto va comprimido, o un recuento sobre el texto de una salida cuando lo que se pretendía medir era un árbol de dependencias. La segunda es **un documento que envejece en horas**: un pendiente describe lo que alguien entendió al escribirlo, no lo que el código hace ahora, y fiarse de él produjo tres pendientes caducados en una sola respuesta.
- **Y la limitación de fondo, que explica una norma de trabajo entera:** el asistente **no recuerda** las sesiones anteriores. Lo que no quede escrito en el diario o en esta memoria no existe para la sesión siguiente. De ahí que el diario se escriba **en el momento** y no al cerrar la fase: no es disciplina, es la única forma de que el motivo de una decisión sobreviva a quien la tomó.

Lo que el registro de horas no puede decir

El diario tiene **176 entradas**, **174** con el campo de horas, y suma **85,9 h registradas**. Pero el campo **solo empezó a nombrar a quién** en el tramo final:

- **39,5 h atribuidas** explícitamente al asistente, en 99 entradas.
- **38,0 h de las Fases 0, 1 y 2**, más 8,5 h de entradas anteriores a la numeración por fases, **sin dueño anotado**.

Por eso el apartado 10 **no puede repartir esas horas por persona a partir de este registro**, y conviene decirlo en lugar de rellenar una tabla que parecería medida. Caben dos salidas honestas: aplicar el modelo de trabajo vigente, el asistente construyó, de modo que la mayor parte de esas horas serían suyas, o declararlas no atribuibles. **Esta memoria no lo resuelve por su cuenta:** lo anota como lo que es, un límite del registro, y deja la decisión a quien firma el apartado 10. Lo que sí está descartado es usar el número de confirmaciones como reparto, por el motivo que ese mismo apartado explica: la autoría se sortea.

SECCIÓN 12

Anexos

Manual de instalación ampliado, credenciales de prueba (ficticias), glosario y referencias.

Manual de despliegue

El despliegue es **manual y deliberadamente aburrido**: la instalación pública solo recibe versiones, nunca arreglos. Está prohibido editar ficheros en ella, fue la causa de que al principio de la práctica lo desplegado no estuviera en ninguna rama, y por eso la actualización se hace con **traer y reponer a la fuerza** el estado de la rama principal, no con una fusión: si alguien hubiera editado algo allí, el siguiente despliegue se lo lleva. Es intencionado.

Orden de los pasos, y por qué este y no otro:

1. **Punto de retorno antes de tocar nada:** etiquetar las imágenes en servicio y copiar la configuración. No cambia nada y es lo que permite volver en un minuto.
2. **Fusionar la rama de trabajo en la principal** mediante solicitud de incorporación. La rama principal está protegida y la solicitud sale bloqueada por la regla de revisión. La desbloquea quien administra.
3. **Desplegar:** traer y reponer a la fuerza, y levantar **solo los servicios que el cambio toca**.
4. **Verificar con sondas,** nunca por impresión visual.
5. **Si falla,** volver al punto 1.
6. **Etiquetar** la versión entregada.

Qué servicios se levantan, y esto se mide, no se supone. Se listan los ficheros que cambian entre las dos ramas y se agrupan por carpeta. En el despliegue de la congelación: tres servicios se reconstruyeron (interfaz, API y módulo de IA), uno se **recreó sin reconstruir**, solo ganaba una línea de red, y su imagen pesa 2,4 GB, y dos no se tocaron porque ni su imagen ni su configuración cambiaban. Dos lecciones que costaron tiempo y conviene no reaprender:

- **Nombrar un servicio no basta para recrearlo.** Levantar con reconstrucción un servicio cuya imagen se descarga y cuyo bloque no cambió **no lo recrea**: el despliegue sale «en verde» sin aplicar la configuración nueva. Para eso hace falta pedir la recreación explícitamente.
- **Quitar un servicio del fichero de composición no para su contenedor:** queda **huérfano** y sigue corriendo. Para que desaparezca de verdad hay que pedir que se retiren los huérfanos, y eso es parar y eliminar un contenedor, así que lo decide y lo lanza una persona, no el guion.

Verificación por sondas. Cada sonda tiene un valor esperado **medido antes en el entorno de trabajo**, y la lista incluye **un control negativo**: un marcador que debe dar cero. Sin ese control, una sonda que no encuentra nada es indistinguible de una sonda mal apuntada. Dos de las sondas de la primera versión estaban mal dirigidas, preguntaban a la pieza equivocada, y se cazaron **probándolas**, que es para lo que se prueban.

Avisos que no son opcionales:

- La reposición a la fuerza **no toca el fichero de entorno** (está excluido del control de versiones): la instalación pública conserva su clave y su código de invitación.
- Las variables nuevas traen valor por omisión, así que no hay que tocar el entorno para desplegar. Pero el tope de tokens por omisión **se queda corto** con el modelo actual: la

respuesta se trunca y el análisis sale como «no analizado» (RNF-06). **Y subirlo no es la salida**: el tope por omisión está justo por debajo del límite que el modelo admite sin entregar la respuesta por partes, y el cliente no la pide por partes en ninguna llamada, así que al subirlo la consulta falla con un error explícito. Lo que queda, mientras no se añada esa entrega por partes, es convivir con el truncado: el análisis sale **no analizado** con su motivo, que es justo lo que RNF-06 pide. El propio producto recomienda subirlo en su mensaje de error, y ese consejo **también** hay que corregirlo.
- La reconstrucción de la interfaz es la pesada: si queda poca memoria libre, conviene saberlo antes.

Usuarios de prueba y acceso para la corrección

El producto ya no tiene una contraseña compartida: desde el cambio de acceso cada persona **se da de alta ella misma** con un código de invitación y elige su contraseña, que nadie más conoce. Eso cambia cómo se entrega el acceso para corregir, y conviene decirlo con claridad:

- **En este repositorio no hay ninguna credencial**, ni de prueba ni ficticia. El repositorio es **público**: una contraseña escrita aquí sería una contraseña publicada, y la instalación a la que abre está en internet.
- **Las credenciales de corrección se entregan por el canal del máster**, junto a la dirección de la instalación. Incluyen lo necesario para entrar y, si se quiere crear una cuenta propia, el código de invitación.
- **Las cuentas que se usaron para probar** durante el desarrollo viven en el entorno de trabajo, no en la instalación pública, y se retiran antes de la entrega. Se nombraron con prefijos reconocibles por propósito, alta, rastreo, víctima y atacante, precisamente para poder localizarlas y borrarlas luego.
- **Qué se puede comprobar con una sola cuenta:** todo el recorrido de auditoría. Para comprobar el **aislamiento** hacen falta dos, y así se verificó: once de once comprobaciones con dos cuentas distintas.

Glosario

- **Spider (rastreador):** recorre un sitio siguiendo enlaces y formularios para levantar su mapa.
- **Repeater:** banco de trabajo de **una** petición: se modifica y se reenvía tal como se ve en pantalla.
- **Intruder:** prueba muchos valores en un punto marcado de la petición (fuzzing) y señala las respuestas que se salen del patrón.
- **Proxy del lado servidor:** el producto hace la petición **en lugar** del navegador, de modo que no hace falta instalar ni configurar nada en el equipo de quien audita.
- **Fichero de autoconfiguración de proxy:** mecanismo de la primera versión para que el navegador enrutara solo. Se jubiló al rehacer el acceso, junto con sus rutas.
- **Token firmado (JWT):** credencial que emite el servidor y que el cliente presenta en cada petición. Aquí el algoritmo de firma está fijado en el servidor y **no se lee del propio token**.
- **Hash de contraseña:** huella irreversible. Lo que se guarda es la huella, nunca la contraseña.
- **Bus interno:** canal por el que los servicios se hablan sin exponerse a internet. Sustituye a que un módulo llame a la API como si fuera un cliente más.
- **Operación degradada:** seguir dando servicio con menos garantías cuando una pieza falla, **y que se note**. Un fallo silencioso que se disfraza de resultado correcto es peor que una caída.

- **Techo de gasto:** límite de llamadas al modelo por auditoría. Al agotarse, el resultado sale degradado y visible en lugar de silencioso.
- **Trampa de araña:** sitio que genera direcciones infinitas y hace que un rastreador no termine nunca.
- **Fuga de aislamiento:** que los datos de un usuario aparezcan en la sesión de otro.
- **Huella digital (fingerprint):** identificar la tecnología de un objetivo por sus respuestas.
- **Entorno de trabajo y instalación pública:** las dos instalaciones del producto. Todo se monta y se prueba en la primera. La segunda solo recibe versiones (apartado 4).
- **Contenedor huérfano:** el que sigue corriendo después de que su definición desapareciera del fichero de composición.
- **RF / RNF:** requisito funcional (lo que el producto hace) y no funcional (cómo lo hace: seguridad, rendimiento, despliegue).
- **STRIDE:** modelo de amenazas que recorre seis familias, suplantación, manipulación, repudio, divulgación, denegación de servicio y elevación de privilegio, para no olvidar ninguna (apartado 6).

Referencias

Fuentes del enunciado y de la práctica anterior (material del máster, no redistribuido en este repositorio): el enunciado de esta práctica, el de la práctica anterior y el informe que el grupo entregó entonces. De ahí salen la estructura obligatoria de trece apartados, la numeración de requisitos que la práctica anterior no tenía y el plan de mejoras cuyos cinco primeros puntos delimitan el alcance (apartado 3).

Dentro de este repositorio, y todas estas rutas existen, se comprobó al escribirlas: docs/informe/P3-memoria.typ (fuente única de esta memoria), docs/informe/diario.md (registro de trabajo con el motivo de cada decisión, las horas y lo que se descartó), docs/capturas/ (figuras, con el nombre atado al requisito que demuestran) y docs/evidencias/ (salidas de terminal guardadas como texto, no como foto, para que sean buscables y comparables).

Taxonomías y modelos usados: la clasificación de severidad que pide RF-08 se apoya en la taxonomía de OWASP, y el modelo de amenazas del apartado 6 sigue STRIDE.

Herramientas de medición citadas en la memoria, para que cualquiera pueda reproducir las cifras: el auditor de dependencias de cada ecosistema (el del gestor de paquetes de la interfaz y pip-audit para los módulos en Python), pdftotext para escanear el PDF entregable en busca de secretos, sobre el PDF crudo el texto va comprimido y la búsqueda daría cero siempre, y el compilador de documentos con el que se generan las dos salidas de esta memoria.

Material de trabajo del grupo que no vive en este repositorio (protocolo de trabajo, flujo de ramas, procedimiento de despliegue y planes por fases) y que se cita en la memoria por su contenido, no por su ruta, precisamente para no dejar referencias que un lector del repositorio no podría abrir.